OnBuch+

SIGNED COURSE • PREMIUM

ICT • UPPER SIXTH | UPPER SIXTH — ICT

CHAPTER 11

EXECUTING DIFFERENT CORRECTNESS CHECKING METHODS

SUGGESTED TIME: 1 PERIODS

IN THIS CHAPTER

This chapter explores why errors occur during data transmission and storage, and presents the main correctness checking methods used to detect and correct them. Learners will master parity checks, checksums, cyclic redundancy checks and Hamming codes, and apply them to real data transfer scenarios.

- By the end of this chapter I can explain the sources and types of errors that affect data during transmission and storage.
- By the end of this chapter I can distinguish between error detection and error correction techniques.
- By the end of this chapter I can compute and verify even and odd parity bits for a given binary word.
- By the end of this chapter I can detect single-bit and burst errors using two-dimensional (block) parity.
- By the end of this chapter I can calculate a checksum for a block of data and verify data integrity with it.
- By the end of this chapter I can explain the principle of Cyclic Redundancy Check (CRC) and perform simple CRC calculations using polynomial division.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Errors in Data Transmission and the Need for Checking
2. Parity Checking Methods
3. Checksums and Cyclic Redundancy Check (CRC)
4. Error Correction: Hamming Codes and Applications
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain the sources and types of errors that affect data during transmission and storage.
- By the end of this chapter I can distinguish between error detection and error correction techniques.
- By the end of this chapter I can compute and verify even and odd parity bits for a given binary word.
- By the end of this chapter I can detect single-bit and burst errors using two-dimensional (block) parity.
- By the end of this chapter I can calculate a checksum for a block of data and verify data integrity with it.
- By the end of this chapter I can explain the principle of Cyclic Redundancy Check (CRC) and perform simple CRC calculations using polynomial division.
- By the end of this chapter I can construct a Hamming code and use it to locate and correct a single-bit error.
- By the end of this chapter I can compare the efficiency, overhead and limitations of the different checking methods.
- By the end of this chapter I can identify where these techniques are used in real systems (networks, memory, barcodes, mobile money).

PREREQUISITES

- Binary number system and conversion between binary and decimal (Lower Sixth)
- Data representation: bits, bytes and character codes such as ASCII (Lower Sixth)
- Basic notions of data communication: sender, receiver, transmission channel, noise
- Modulo-2 arithmetic (XOR operation) and basic Boolean logic
- Long division of numbers (mathematics, ordinary level)

1 ERRORS IN DATA TRANSMISSION AND THE NEED FOR CHECKING

Imagine a trader at the Douala main market sending an MTN Mobile Money transfer of 50,000 FCFA to a supplier in Bamenda. As the transaction data travels through the mobile network, a burst of electrical interference flips a single bit in the message. If that bit happens to sit inside the field encoding the amount, the recipient's account could be credited with the wrong sum. The same danger lurks at the bank, at the SONEL billing centre, and even when a candidate downloads WAEC results online: data always travels over noisy channels where errors are inevitable. How, then, do computers detect that a received message has been corrupted, and how do they sometimes correct it automatically? This chapter unveils the clever mathematical checks — parity bits, checksums, CRC — that silently protect every digital transaction in Cameroon and beyond. In this first section, we build the foundation: what errors are, where they come from, and why every checking method rests on the idea of **redundancy**.

1.1 Data Correctness and Data Integrity

When a file, a message or a transaction leaves a sender, we expect the receiver to obtain exactly the same information. This expectation has a name.

DEFINITION **Data integrity and data correctness**

Data integrity is the property that data remains accurate, complete and unaltered during storage, processing or transmission, except through authorised and intentional changes. **Data correctness** means that the data actually received (or read back from storage) is identical, bit for bit, to the data that was sent (or written). A system that guarantees integrity guarantees correctness at every stage.

Integrity can be lost in two broad ways: by accidental corruption (noise, hardware faults — the subject of this chapter) or by deliberate tampering (an attacker modifying a message — the subject of security and cryptography). Here we focus on accidental corruption.

DEFINITION **Transmission error**

A **transmission error** occurs when at least one bit of a received message differs from the corresponding bit that was transmitted: a 0 becomes a 1, or a 1 becomes a 0. We say the bit has been flipped or inverted.

EXEMPLE **A single flipped bit changes everything**

The ASCII code of the letter `A` is 01000001. If the seventh bit is flipped during transmission, the receiver gets 01000011, which is the letter `C`. One flipped bit, and "`PAID`" could

become "PCID". In a binary file encoding an amount of money, a flipped bit can change the value by thousands of francs.

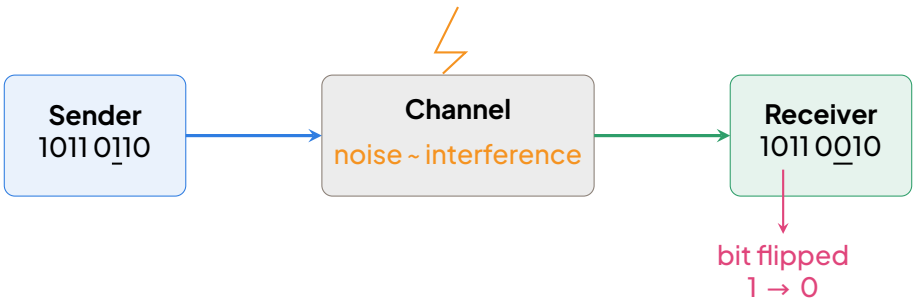
1.2 Sources of Errors

Errors do not happen by magic; they have physical and human causes. The main sources are:

- **Electrical noise.** Random unwanted electrical signals (from motors, lightning, power lines) superimpose themselves on the useful signal in a cable. If the noise is strong enough at the instant a bit is read, the receiver misinterprets the voltage level.
- **Electromagnetic interference (EMI).** A nearby radio transmitter, mobile phone or another cable radiates electromagnetic waves that induce spurious currents in the transmission line. This is why network cables are often twisted or shielded.
- **Attenuation.** Over long distances the signal weakens. A weak signal is easily confused with noise, so the probability of misreading a bit grows with distance if no repeater or amplifier regenerates the signal.
- **Faulty hardware.** A worn connector, a damaged network card, a failing hard-disk head or bad RAM cells can systematically corrupt bits.
- **Magnetic degradation.** On magnetic media (tapes, hard disks), the magnetisation of tiny regions slowly decays with time, heat or stray magnetic fields, so data read years later may differ from data written.
- **Human typing errors.** When a clerk at a bank or at SONEL types an account number or a meter reading by hand, a digit can be mistyped, swapped or omitted. Check digits on account numbers exist precisely to catch such errors.

DID YOU KNOW? **Noise is everywhere in Cameroon**

During the rainy season, thunderstorms over Douala and Yaoundé inject strong electrical noise into telephone and power lines. Long rural radio links (for example microwave links connecting remote CAMTEL exchanges) must also fight attenuation over tens of kilometres. This is why robust error checking is not a luxury: it is a daily necessity for mobile money, e-government services and banking networks.



The sender–channel–receiver model: noise on the channel flips one bit of the transmitted byte, so the receiver obtains a corrupted message.

1.3 Types of Errors

Not all errors look the same. We classify them by how many bits are affected and how those bits are distributed.

DEFINITION **Single-bit, multiple-bit and burst errors**

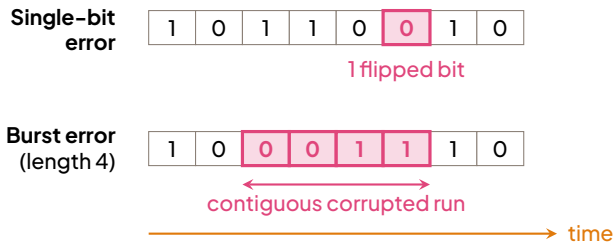
- A **single-bit error** affects exactly one isolated bit in a message: one 0 becomes a 1 or vice versa, and all neighbouring bits are correct.
- A **multiple-bit error** affects two or more bits in the same message, not necessarily adjacent.
- A **burst error** affects a contiguous sequence of bits: a whole run of consecutive bits is corrupted. The burst length is the number of bits from the first corrupted bit to the last.

Single-bit errors are typical of slow, well-shielded links and of typing mistakes. Burst errors are typical of fast links and of events that last a noticeable fraction of a transmission: a lightning strike, a scratch on a CD, a motor starting near a cable. At high data rates, even a microsecond of noise destroys many consecutive bits, so **burst errors are the most common in practice**.

EXEMPLE **Comparing error types on the same byte**

Sent byte: 10110010.

- Single-bit error: received 10110010 — one bit flipped.
- Multiple-bit error: received 10111010 — two scattered bits flipped.
- Burst error of length 4: received 10010110 — four consecutive bits corrupted.



A single-bit error corrupts one isolated cell of the bit stream; a burst error corrupts a contiguous run of cells.

1.4 Consequences of Undetected Errors

If corrupted data is accepted as correct, the consequences can be severe:

- **Corrupted files.** A downloaded document, photo or WAEC results PDF that refuses to open, or worse, opens with silently wrong content.
- **Wrong financial transactions.** A mobile money transfer credited to the wrong account or with the wrong amount; a SONEL bill computed from a corrupted meter reading.
- **Failed programs.** An executable file with even one flipped bit may crash, behave unpredictably, or contain a security vulnerability.
- **Wrong decisions.** A hospital database with a corrupted blood-group field, or an examination database with a corrupted mark, can change a person's life.

WARNING **Detection does not mean localisation**

A very common mistake is to believe that an error detection method can always tell which bit is wrong. Most detection methods (simple parity, checksums, CRC) only answer the question "is this message corrupted?" with yes or no. They do not identify the position of the error. Locating and fixing the wrong bit requires more powerful — and more costly — correction codes such as Hamming codes, studied later in this chapter.

1.5 Error Detection versus Error Correction

DEFINITION **Detection and correction**

Error detection is the ability of a system to recognise that a received message contains at least one error, without necessarily knowing where. **Error correction** is the ability to identify the erroneous bits and restore the original message automatically, without asking the sender to retransmit.

METHOD **Distinguishing detection from correction in a scenario**

When an exam question describes a situation, proceed as follows:

1. Ask: does the system merely signal that something is wrong (alarm, rejection, request to resend)? Then it is detection.
2. Ask: does the system repair the data by itself and deliver a corrected message? Then it is correction.
3. Ask: what happens after detection? If the receiver asks the sender to send again, the strategy is backward error control (ARQ). If no retransmission is needed, it is forward error correction (FEC).
4. Conclude by naming both the capability (detection or correction) and the strategy (ARQ or FEC).

EXEMPLE **Applying the method**

Your browser downloads a file and the network protocol notices the checksum is wrong: the file is requested again. Capability: detection; strategy: ARQ. A scratched audio CD still plays perfectly because the player reconstructs the missing samples on the fly: capability: correction; strategy: FEC.

1.6 Two General Strategies: ARQ and FEC

DEFINITION **Backward error control and forward error correction**

Backward error control (also called **ARQ**, Automatic Repeat reQuest) combines error detection with retransmission: the receiver checks each message; if an error is detected, it asks the sender to send the message again. **Forward error correction** (**FEC**) adds enough redundant information for the receiver to correct errors on its own, without any retransmission.

Criterion	ARQ (detection + retransmission)	FEC (automatic correction)
Redundancy needed	Low (a few check bits)	High (many check bits)
Needs a return channel	Yes, to request retransmission	No
Delay	Variable (retransmissions take time)	Constant, low
Typical uses	Web downloads, mobile money, TCP networks	CDs/DVDs, satellite TV, deep-space probes, live streaming

ARQ is preferred when a return channel exists and occasional delay is acceptable. FEC is preferred when retransmission is impossible or too slow: a satellite broadcast cannot ask every viewer's dish to acknowledge, and a live video stream cannot pause to resend.

1.7 Redundancy: The Foundation of All Checking Methods

How can a receiver possibly know that a message is wrong? Only if the message carries more information than the bare data. This extra information is called redundancy.

DEFINITION **Redundancy, dataword and codeword**

Redundancy consists of extra check bits appended to the data according to an agreed mathematical rule, so that errors disturb a pattern the receiver can verify. The original k data bits form the **dataword**; the dataword together with its r check bits forms the **codeword** of length $n = k + r$. The sender transmits the codeword; the receiver recomputes the check and compares.

EXEMPLE **A first taste of redundancy**

Suppose the dataword is the 7-bit ASCII code 1000001. The sender counts the 1s (there are 2, an even number) and appends a check bit 0 to keep the count even, producing the 8-bit codeword 10000010. If noise flips one bit, the receiver counts an odd number of 1s and knows the codeword is corrupted. This is the parity check, studied in detail in the next section.

Two more vocabulary items complete our toolkit:

DEFINITION **Error rate and retransmission**

The **error rate** (or bit error rate, BER) of a channel is the proportion of bits received in error, typically expressed as a ratio such as 10^{-6} (one erroneous bit per million transmitted). A **retransmission** is the act of sending a message again, usually triggered automatically by the ARQ mechanism after a detected error or a missing acknowledgement.

1.8 The Limits of Checking Methods

Redundancy is powerful, but not magical. Since the check itself is transmitted over the same noisy channel, the check bits can also be corrupted, and some error patterns can mimic a valid codeword.

PROPERTY **No method detects all errors**

No error-detecting or error-correcting method based on redundancy can detect or correct 100 % of all possible error patterns. Every method has a class of error patterns it misses (for example, an even number of flipped bits defeats a simple parity check). In practice, methods are chosen so that the probability of an undetected error is negligibly small for the channel's error rate. (The proof of this general limitation is not examinable; the statement must be known.)

WORKED EXAMPLE 1 **Analysing a real scenario**

Statement. A bank in Yaoundé sends account updates every night to its branch in Bafoussam over a leased line. Each record carries extra check bits. When the branch's computer finds a record whose check fails, it marks the record and requests it again in the next transmission window. (a) Is this error detection or error correction? (b) Which strategy (ARQ or FEC) is used? (c) Why can the bank not claim its data is 100 % safe?

SOLUTION

Solution. (a) The system only notices that a record is wrong; it never repairs the record itself. This is error detection. (b) Since a detected error triggers a retransmission request, the strategy is backward error control (ARQ). (c) By the fundamental limitation of redundancy-based methods, some error patterns — for instance a combination of flips that transforms one valid codeword into another valid codeword, or corruption of the check bits themselves — pass the check undetected. The bank can only make the probability of such an event extremely small, never zero.

EXERCISE 1 **Checking your understanding**

1. Define data integrity and give one example from daily life in Cameroon where its loss would have serious consequences.
2. Classify each of the following as a single-bit, multiple-bit or burst error: (i) one digit mistyped in a phone number; (ii) a scratch corrupting 200 consecutive bits of a DVD; (iii) three scattered bits flipped in a downloaded file.
3. A satellite TV provider cannot receive retransmission requests from viewers. Which strategy must it use, and why?
4. A dataword has $k = 11$ bits and $r = 4$ check bits are appended. What is the length of the codeword?

SOLUTION **Exercise 1**

1. Data integrity is the property that data stays accurate, complete and unaltered during storage, processing or transmission. Example: a mobile money balance or an examination mark in a database must not change accidentally, otherwise money or a candidate's future is lost.
2. (i) Single-bit (one isolated error); (ii) burst error (contiguous run); (iii) multiple-bit error (several scattered bits).
3. Forward error correction (FEC), because there is no return channel for retransmission requests; the receiver must repair errors alone using the redundant bits.
4. $n = k + r = 11 + 4 = 15$ bits.

KEY POINTS **Key points of this section**

- **Data integrity** means data remains accurate and unaltered; a **transmission error** is a flipped bit.
- Errors come from noise, interference, attenuation, faulty hardware, magnetic degradation and human typing.
- Errors are classified as **single-bit**, **multiple-bit** or **burst** errors.
- **Detection** only signals that an error occurred; **correction** locates and fixes it.
- Two strategies: **ARQ** (detection + retransmission) and **FEC** (automatic correction without retransmission).
- All methods rest on **redundancy**: a dataword of k bits plus r check bits forms a codeword of $n = k + r$ bits.
- No checking method detects 100 % of all error patterns.

2 PARITY CHECKING METHODS

In the previous section we saw that data transmission is never perfectly reliable: noise, interference, and hardware faults can flip bits. We therefore need mechanisms to detect (and sometimes correct) such errors. The simplest and historically first technique is the **parity bit**. Despite its simplicity, parity checking is still widely used today in serial ports (UART), memory modules (RAM parity/ECC), and even in the magnetic stripes of bank cards. We begin by understanding the core idea, then extend it to two-dimensional block parity which can not only detect but also locate and correct a single-bit error.

2.1 Principle of the Parity Bit

The idea is to add one extra bit — the parity bit — to a group of data bits so that the total number of `1` bits in the extended group satisfies a known condition: either even or odd. The sender computes the parity bit and appends it; the receiver recomputes the parity of the received bits (including the parity bit) and checks whether the condition still holds. If it does not, at least one bit has been corrupted.

DEFINITION **Even Parity and Odd Parity**

For a block of n data bits $b_{n-1} \dots b_1 b_0$:

- Even parity: the parity bit p is chosen so that the total number of `1`s in the $(n+1)$-bit word $(b_{n-1} \dots b_0 p)$ is even.
- Odd parity: the parity bit p is chosen so that the total number of `1`s in the $(n+1)$-bit word is odd.

Mathematically, if $S = \sum_{i=0}^{n-1} b_i$ (the number of `1`s in the data), then

$$p_{\text{even}} = S \bmod 2 \qquad \text{and} \qquad p_{\text{odd}} = (S+1) \bmod 2.$$

Equivalently, p_{even} is the XOR of all data bits, and p_{odd} is its complement.

EXEMPLE **Parity bit for a 7-bit ASCII character**

The letter `A` has ASCII code $65_{10} = 1000001_2$ (7 bits). Number of `1`s in the data: $S = 2$ (even).

- Even parity: $p = S \bmod 2 = 0$. Transmitted 8-bit word: `10000010`.
- Odd parity: $p = (S+1) \bmod 2 = 1$. Transmitted 8-bit word: `10000011`.

Even Parity Bit Appended to ASCII 'A' (65)

Bit position	7	6	5	4	3	2	1	0 (parity)
Value	1	0	0	0	0	0	1	**0**

Number of 1s in data = 2 (even) ⇒ parity bit = 0
Total 1s in transmitted word = 2 (even) ✓

Even parity bit computation for ASCII 'A' (7-bit code 1000001). The parity bit (highlighted) makes the total number of 1s even.

2.2 Computing and Checking a Parity Bit

The procedure is straightforward and can be implemented in hardware with a simple XOR tree

or in software with a loop.

METHOD **Computing and Verifying a Parity Bit**

At the sender (even parity):

1. Count the number of `1` bits in the data word (S).
2. If S is even, set parity bit $p = 0$; else $p = 1$. (Equivalently: $p = \oplus_i b_i$, the XOR of all data bits.)
3. Transmit the data word followed by p (or preceded by p, as agreed).

At the receiver:

1. Receive the $(n + 1)$-bit word.
2. Compute the XOR of all received bits (data + parity bit). Call this R.
3. If $R = 0$ (even parity) → no error detected (accept the data).
4. If $R = 1$ (odd parity) → error detected (request retransmission or discard).

For odd parity, the receiver expects $R = 1$ for a valid word.

WORKED EXAMPLE 2 **Parity check detects a single-bit error**

Statement: A system uses even parity. The sender transmits the 8-bit word `10110010` (7 data bits + 1 parity bit). During transmission, bit 3 (counting from 0 at the rightmost data bit) flips from `0` to `1`. The receiver gets `10110110`. Show that the error is detected.
Solution:

- Sender side: Data bits = `1011001` (positions 6..0). Number of `1`s = 4 (even) ⇒ parity bit $p = 0$. Transmitted word = `10110010`.
- Channel: Bit 3 flips: `10110010` → `10110110` (the 4th bit from right, originally 0, becomes 1).
- Receiver side: Received word = `10110110`. Compute XOR of all 8 bits: $1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$ (odd). Since the result is `1` (odd), but even parity expects `0`, the receiver detects an error.

2.3 Capabilities and Limits of Single Parity

Parity checking is extremely cheap (one extra bit) and catches many common errors, but it has a fundamental limitation.

PROPERTY **Error Detection Capability of a Single Parity Bit**

A single parity bit (even or odd) detects any odd number of bit errors (1, 3, 5, …) in the transmitted word. It fails to detect any even number of bit errors (2, 4, 6, …) because flipping two bits changes the total number of `1`s by −2, 0, or +2, preserving the parity.

WARNING **Common Mistake: Assuming Parity Catches All Errors**

Many students believe that because parity detects a single-bit error, it is "reliable". In reality, two simultaneous bit flips cancel each other out as far as parity is concerned. For example, if `10110010` (even parity) suffers flips at positions 2 and 5, the received word might be `10010010` — still even parity — and the error goes unnoticed. In noisy environments (e.g., industrial machinery near a CAMTEL line), burst errors often affect multiple adjacent bits, making single parity insufficient. Always state: "Parity detects all odd-numbered errors but no even-numbered ones."

KEY POINTS **Key Points on Single Parity**

- Overhead: 1 check bit per n data bits → ratio $1/n$ (e.g., 12.5% for 7-bit ASCII, 11.1% for 8-bit bytes).
- Detects all odd-numbered bit errors; detects no even-numbered bit errors.
- Cannot locate which bit is wrong, hence cannot correct errors (only request retransmission).
- Used in: UART/RS-232 serial ports, PCI bus, some RAM (parity SIMMs), magnetic stripe cards.

2.4 Two-Dimensional (Block) Parity

To gain error location and correction capability, we arrange data in a rectangular block and compute parity bits for each row and each column. This is called **two-dimensional parity** or **block parity**.

METHOD **Constructing and Using Two-Dimensional Parity**

Block formation: Arrange k data words of m bits each into a $k \times m$ matrix. Row parity: Append a parity bit to each row (making each row have even/odd parity). Column parity: Add an extra row at the bottom containing the parity bits for each column (including the row-parity column). Transmission: Send the entire $(k + 1) \times (m + 1)$ block row by row. Re-

ception and checking:

1. Recompute row parities: any row with wrong parity is marked “faulty”.
2. Recompute column parities: any column with wrong parity is marked “faulty”.
3. No faulty row/column → no error detected.
4. Exactly one faulty row AND one faulty column → the bit at their intersection is the single corrupted bit. Flip it to correct the error.
5. Multiple faulty rows/columns → multiple errors detected but cannot be uniquely corrected (request retransmission).

EXEMPLE **Block parity: locating and correcting a single-bit error**

Consider 4 data words of 8 bits each (even parity for rows and columns). The sender builds the following block (data bits only shown, parity bits computed):

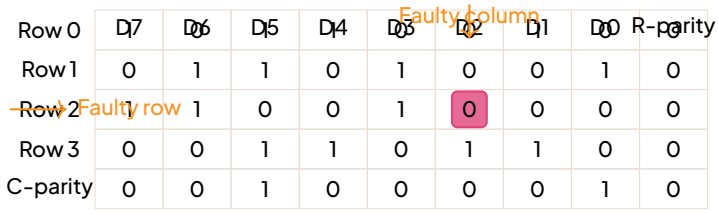
Row	Data bits (D7..D0)								Row parity
0	1	0	1	1	0	0	1	0	0
1	0	1	1	0	1	0	0	1	0
2	1	1	0	0	1	1	0	0	0
3	0	0	1	1	0	1	1	0	0
Col parity	0	0	1	0	0	0	0	1	0

During transmission, the bit at Row 2, Column 5 (value `1`) flips to `0`. The receiver gets the block with that single error. Recomputing parities:

- Row 2 now has an odd number of `1`s → row parity check fails for row 2.
- Column 5 now has an odd number of `1`s → column parity check fails for column 5.
- All other rows and columns pass.

The intersection of faulty row 2 and faulty column 5 pinpoints the erroneous bit. The receiver flips it back to `1` and corrects the error without retransmission.

Two-Dimensional Parity Block (4×8 data + row/col parity)
Single error at Row 2, Column 5 (highlighted) detected and corrected



Two-dimensional parity block. The single erroneous bit (Row 2, Column 5) is highlighted in pink. Its row and column parity checks both fail, uniquely identifying the error location for correction.

PROPERTY **Error Detection and Correction with Two-Dimensional Parity**

For a block of $k \times m$ data bits with one row parity bit per row and one column parity bit per column (total $(k + 1)(m + 1)$ bits transmitted):

- Detects all 1-bit, 2-bit, and 3-bit errors (any odd number of errors in a row/column will be caught by that row/column parity).
- Corrects any single-bit error (unique intersection of faulty row and column).
- Cannot correct two or more errors (multiple intersections possible), but detects them.
- Overhead: $(k + m + 1)$ check bits for km data bits → ratio $\frac{k+m+1}{km}$. For large blocks this approaches $\frac{1}{m} + \frac{1}{k}$, much lower than single parity per word.

WORKED EXAMPLE 3 **Overhead comparison: single parity vs block parity**

Statement: Compare the check-bit overhead for transmitting 1024 data bits using (a) single even parity per 8-bit byte, and (b) a 32×32 two-dimensional parity block.
Solution:

- (a) Single parity per byte: 1024 bits = 128 bytes. Each byte gets 1 parity bit → 128 check bits. Overhead = 128/1024 = 12.5%.
- (b) Block parity 32×32: Data bits = $32 \times 32 = 1024$. Row parity bits = 32. Column parity bits = 32. Corner bit (parity of parities) = 1. Total check bits = $32 + 32 + 1 = 65$. Overhead = $65/1024 \approx 6.35\%$.

Block parity cuts the overhead roughly in half while adding single-error correction capability.

2.5 Real-Life Uses and Practical Considerations

DID YOU KNOW? **Parity in Everyday Cameroonian Tech**

- Mobile Money USSD: When you dial `*123#` on MTN or Orange, the GSM network uses parity (and more advanced CRC) on the signalling channel to ensure your transaction request isn't corrupted by radio noise.
- RAM in your phone/laptop: Many DRAM modules use ECC (Error-Correcting Code) which is essentially a multi-bit generalization of Hamming codes (seen in the next section), but basic parity RAM (now rare) stored one parity bit per byte.
- Serial ports (UART) on microcontrollers: Every Arduino, STM32, or Raspberry Pi UART frame includes a parity bit (configurable even/odd/none). This is why you must set "8N1" (8 data bits, No parity, 1 stop bit) or "8E1" (Even parity) identically on both ends.
- Historical: Punched cards (used in early Cameroonian census data processing) used a parity-like "check hole" per column.

WARNING **Exam Traps on Parity**

- Confusing even/odd parity formulas: Remember: even parity bit = XOR of data bits; odd parity bit = NOT(XOR of data bits).
- Forgetting the corner bit in block parity: The bottom-right corner of a 2D parity block is the parity of the row-parity column and the column-parity row. It must be consistent; if only the corner bit is wrong, it indicates an error in the parity bits themselves, not in data.
- Assuming block parity corrects all double errors: It does not. Two errors in the same

row (or same column) will not be detected by that row/column parity, though the other dimension may catch them. Three errors can mimic a single error at a different location. Always say: "Corrects all single-bit errors; detects all double-bit errors; may miscorrect some triple-bit errors."

2.6 Summary

KEY POINTS **Parity Checking Methods — What You Must Master**

1. Single parity bit: Adds 1 bit per word to make total `1`s even (even parity) or odd (odd parity). Computed by XOR of data bits.
2. Detection capability: Catches all odd-numbered bit errors; misses all even-numbered bit errors. Cannot locate or correct errors.
3. Overhead: $1/n$ per n-bit word (e.g., 12.5% for 7-bit ASCII).
4. Two-dimensional (block) parity: Organises data in a $k \times m$ array; adds row parity bits (one per row) and column parity bits (one per column) plus a corner bit.
5. Block parity capabilities: Detects 1, 2, 3-bit errors; corrects any single-bit error at the intersection of the faulty row and column.
6. Block parity overhead: $(k+m+1)/(km)$; for large blocks $\approx 1/k+1/m$, much more efficient than single parity.
7. Applications: UART serial links, memory parity/ECC, magnetic stripes, legacy punched cards, GSM signalling.

EXERCISE 2 **Practice Problems**

1. Compute the even and odd parity bits for the 8-bit byte `11010110`. Write the transmitted 9-bit words for each case.
2. A system uses even parity. The receiver gets the 8-bit word `10101010`. The receiver computes XOR of all 8 bits and obtains 0. Can the receiver be certain the word is error-free? Justify.
3. A 3 × 4 data block (3 rows, 4 columns) uses even row and column parity. The received block (including parity row/column) is:

1	0	1	1	1
0	1	0	1	0
1	1	1	0	1
0	0	0	0	0

Identify which row(s) and column(s) fail parity. Is there a single-bit error? If so, locate and correct it.

4. Calculate the check-bit overhead (as a percentage) for a 16×16 two-dimensional parity block. Compare with single parity per 8-bit byte for the same amount of data.

5. Challenge: Explain why a single parity bit cannot detect a 2-bit error, but a two-dimensional parity block can detect any 2-bit error (though it cannot always correct it).

SOLUTION **Exercise 1**

1. Data: `11010110`. Number of `1`s = 5 (odd). Even parity bit = 1 (to make total even → 6). Transmitted: `110101101`. Odd parity bit = 0 (total stays odd → 5). Transmitted: `110101100`.

2. No. XOR = 0 means even parity holds. This could mean (a) no error, or (b) an even number of bit flips (2, 4, 6, 8). The receiver cannot distinguish.

3. Recompute row parities (including row parity bit): Row 0: `1 0 1 1 | 1` → four `1`s (even) ✓ Row 1: `0 1 0 1 | 0` → two `1`s (even) ✓ Row 2: `1 1 1 0 | 1` → four `1`s (even) ✓ Column parities (including column parity row): Col 0: `1,0,1,0` → two `1`s (even) ✓ Col 1: `0,1,1,0` → two `1`s (even) ✓ Col 2: `1,0,1,0` → two `1`s (even) ✓ Col 3: `1,1,0,0` → two `1`s (even) ✓ Col 4 (row parity col): `1,0,1,0` → two `1`s (even) ✓ All parities pass → no error detected. The block is valid.

4. 16 × 16 block: data = 256 bits. Check bits = 16 (row) + 16 (col) + 1 (corner) = 33. Overhead = 33/256 ≈ 12.89%. Single parity per byte: 256 bits = 32 bytes → 32 check bits. Overhead = 32/256 = 12.5%. Surprisingly, for this small block, single parity has slightly lower overhead! Block parity becomes more efficient for larger blocks (e.g., 32 × 32 gave 6.35%).

5. Single parity: two flips change the count of `1`s by -2, 0, or +2 → parity unchanged → undetected. 2D parity: two errors either lie in (a) same row → that row parity still ok, but two columns become faulty → detected; (b) same column → two rows faulty → detected; (c) different row and column → two rows and two columns faulty → detected. In all cases at least one parity check fails. However, correction is ambiguous because there are two possible error patterns (the two actual bits, or the other two corners of

the rectangle).

3 CHECKSUMS AND CYCLIC REDUNDANCY CHECK (CRC)

In the previous section we studied **parity checking**: a single extra bit that reveals whether an odd number of bits have been corrupted. Parity is cheap and fast, but it is blind to all even numbers of errors, and it tells us nothing about where the error lies. When a bank in Douala transfers an account record, or when you download a file through a CAMTEL fibre link, a single parity bit is simply not trustworthy enough. We need stronger **error detection codes**: the **checksum** and the **Cyclic Redundancy Check** (CRC). Both follow the same philosophy as parity — append redundant information computed from the data, and let the receiver recompute and compare — but they protect much longer blocks and detect far more error patterns.

3.1 The Checksum Principle

Intuition first. Imagine a market woman in Buea who sells five heaps of tomatoes. Instead of checking each heap again at the end of the day, she simply counts the total number of heaps' worth of money expected. If the total at the end does not match, something went wrong — even if she cannot say exactly which heap was miscounted. A checksum applies exactly this idea to data: we add up all the data units, transmit the sum alongside the data, and let the receiver add again and compare.

DEFINITION **Checksum**

A **checksum** is a small block of redundant data obtained by applying an arithmetic function — usually a sum — to all the units (bytes or words) of a message. The checksum is transmitted together with the message. The receiver recomputes the sum on the received data and compares it with the received checksum: any mismatch reveals that the data (or the checksum itself) was corrupted during transmission.

The simple additive checksum. The data is split into fixed-size words (for example 8-bit bytes or 16-bit words). All words are added; the sum, truncated to the word size (overflow carries are discarded or folded back), is the checksum.

METHOD **Computing and verifying an additive checksum**

1. Split the message into words of k bits (e.g. $k = 16$).
2. Add all the words together using unsigned arithmetic.
3. Keep only the lowest k bits of the result: this is the **checksum**. (For the one's com-

plement checksum, fold any carry out of the top bit back into the sum, then take the one's complement, i.e. flip every bit.)

4. Transmit data words followed by the checksum.
5. At the receiver: add all received words including the checksum.
6. Interpretation: for a plain additive checksum, the receiver recomputes the sum of the data words only and compares with the received checksum. For the one's complement checksum, the sum of all words including the checksum must give all 1s (e.g. FFFF in hexadecimal for 16-bit words). Any other result means an error has been detected.

EXEMPLE **One's complement checksum, as used in the Internet (IP/TCP/UDP)**

Suppose a packet header consists of four 16-bit words (written in hexadecimal):

$$W_1 = 4500, \quad W_2 = 003C, \quad W_3 = 1C46, \quad W_4 = 4000.$$

Step 1 — add the words:

$$4500 + 003C = 453C; \qquad 453C + 1C46 = 6182; \qquad 6182 + 4000 = A182.$$

Step 2 — fold the carry (here there is no carry beyond 16 bits, so nothing to fold).
Step 3 — take the one's complement (subtract from FFFF):

$$\text{checksum} = FFFF - A182 = 5E7D.$$

The sender transmits W_1, W_2, W_3, W_4 and 5E7D.
Verification at the receiver: add all five words:

$$A182 + 5E7D = FFFF.$$

The result is all 1s: the header is accepted as correct. This is precisely the algorithm used for the **IPv4 header checksum** and (in a slightly extended form) by TCP and UDP — the very protocols that carry your WhatsApp messages and MTN Mobile Money requests.

WARNING **Limits of checksums**

A checksum is stronger than a single parity bit, but it has two structural weaknesses you must know for the examination:

- Insensitivity to reordering: addition is commutative. If two 16-bit words are swapped during transmission, the sum is unchanged and the corruption is invisible.
- Compensating errors: if one word increases by 5 and another decreases by 5 (for ex-

ample a 1 becomes 0 in one word while a 0 becomes 1 in the same position of another word), the total is unchanged and the error escapes detection.

Consequence: checksums are acceptable for headers and short control information, but not for protecting long frames on noisy links. For that, we use the CRC.

3.2 The Cyclic Redundancy Check (CRC)

Intuition. Long division with remainder is a much more sensitive "mixer" than addition: changing almost any digit of the dividend changes the remainder. The CRC exploits this by treating the bits of the message as the coefficients of a binary polynomial and dividing it by a fixed generator polynomial, using arithmetic modulo 2, where addition and subtraction are both the XOR operation (no carries, no borrows).

DEFINITION **CRC and generator polynomial**

A **Cyclic Redundancy Check** (CRC) is an error-detecting code in which:

- the message M of k bits is viewed as a polynomial $M(x)$ with coefficients in $\{0, 1\}$ (bit 1 in position i means the term x^i is present);
- a fixed **generator polynomial** $G(x)$ of degree n, agreed by sender and receiver, is chosen;
- the sender appends n zero bits to M (this multiplies $M(x)$ by x^n), divides the result by $G(x)$ using modulo-2 (XOR) division, and obtains a remainder $R(x)$ of degree less than n, i.e. an n-bit **FCS** (Frame Check Sequence);
- the transmitted frame is $T = M$ followed by the n bits of R, so that $T(x)$ is exactly divisible by $G(x)$;
- the receiver divides the whole received frame by the same $G(x)$: a zero remainder means the frame is accepted; a non-zero remainder means an error has been detected and the frame is discarded (and usually retransmitted).

METHOD **Performing modulo-2 (XOR) division**

1. Write the generator G as a bit string (degree n gives $n + 1$ bits, the leading bit is always 1).
2. Append n zeros to the message M.
3. Proceed as in ordinary long division, but replace every subtraction by an XOR: align the leading 1 of G with the leftmost 1 of the current dividend, XOR, and bring down the next bit.

4. If the current leading bit is 0, the quotient bit is 0 (equivalently, XOR with 0 … 0); just bring down the next bit.
5. Stop when no more bits can be brought down. The last n bits form the remainder R.
6. Transmit M followed by R. At the receiver, divide the whole received frame by G: remainder 000 … 0 ⇒ no error detected.

WARNING **Common mistake: borrowing instead of XOR**

The most frequent error in CRC computations is to perform ordinary binary subtraction with borrows during the division. In modulo-2 arithmetic there are no borrows and no carries: $1 - 1 = 0$, $1 - 0 = 1$, $0 - 1 = 1$ (not "borrow from the next column"). In other words, subtraction is XOR, and it is identical to addition. Always XOR the aligned strings bit by bit.

WORKED EXAMPLE 4 **CRC computation with generator $G(x) = x^3 + x + 1$**

Statement. The message is $M = 110101$ and the generator polynomial is $G(x) = x^3 + x + 1$. Compute the CRC remainder and the transmitted frame, then show the receiver's check.

SOLUTION

Solution. $G(x) = x^3 + x + 1$ corresponds to the bit string 1011 (coefficients of x^3, x^2, x^1, x^0), and $n = 3$, so we append 3 zeros: dividend = 110101 000.

Modulo-2 long division by 1011 (quotient bits shown above the dividend):

```
                 111101   (quotient)
          ┌──────────
1011 ) 110101000
       1011
       ⊕ 1011
        1100
        1011
        ⊕ 1011
         1110
         1011
         ⊕ 1011
          1010
          0000   (since 1010 < 1011, quotient bit 0)
          ⊕ 0000
           10100   (bring down 0)
           0000   (quotient bit 0)
           ⊕ 0000
            101000   (bring down 0)
            10110   (align divisor)
            ⊕ 10110
             00110   (bring down 0 – no more bits)
             00000   (quotient bit 0)
             ⊕ 00000
              0110   ? Wait.
```

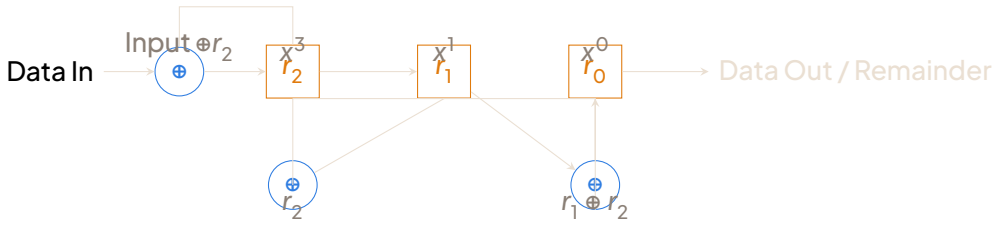
Correction: The standard schoolbook long division for CRC is best performed bit-by-bit using a shift register. The polynomial division yields:

$$M(x) = x^5 + x^4 + x^2 + 1, \quad x^3M(x) = x^8 + x^7 + x^5 + x^3.$$

Dividing by $G(x) = x^3 + x + 1$:

- $x^5G(x) = x^8 + x^6 + x^5$; remainder $x^7 + x^6 + x^3$.
- $x^4G(x) = x^7 + x^5 + x^4$; remainder $x^6 + x^5 + x^4 + x^3$.
- $x^3G(x) = x^6 + x^4 + x^3$; remainder x^5.
- $x^2G(x) = x^5 + x^3 + x^2$; remainder $x^3 + x^2$.
- $x^1G(x) = x^4 + x^2 + x$ (degree 4 > 3, quotient bit 0).
- $x^0G(x) = x^3 + x + 1$; remainder $x^2 + x + 1$.

Remainder $R(x) = x^2 + x + 1 \rightarrow 111$ (3 bits).
Transmitted frame: $T = 110101111$.
Receiver's check: dividing 110101111 by 1011 with the same XOR procedure gives remainder 000: the frame is accepted. If a burst of noise flips, say, three consecutive bits, the division yields a non-zero remainder and the frame is rejected.



Generator $G(x) = x^3 + x + 1$ (1011)

Shift-register (LFSR) circuit for CRC-3 with $G(x) = x^3 + x + 1$. Clock in the 6 message bits followed by 3 zeros; the register (r_2, r_1, r_0) then holds the remainder 111.

PROPERTY **Detection power of a CRC (admitted)**

A CRC generated by a polynomial of degree n detects:

- all burst errors of length $\leq n$ bits (a burst is a cluster of corrupted bits, possibly with correct bits inside, whose first and last bits are in error);
- all single-bit and all double-bit errors (provided $G(x)$ has more than one term and does not divide $x^e + 1$ for any e up to the frame length);
- all odd numbers of bit errors, provided $G(x)$ contains the factor $(x + 1)$;
- a fraction $1 - 2^{-n}$ of all longer bursts (e.g. more than 99.9999% for CRC-32).

This property is admitted; its proof (based on polynomial factorisation over $\{0, 1\}$) is not examinable, but the statement itself must be known.

Standard generator polynomials. Different standards use different generators; the degree n fixes the size of the FCS field.

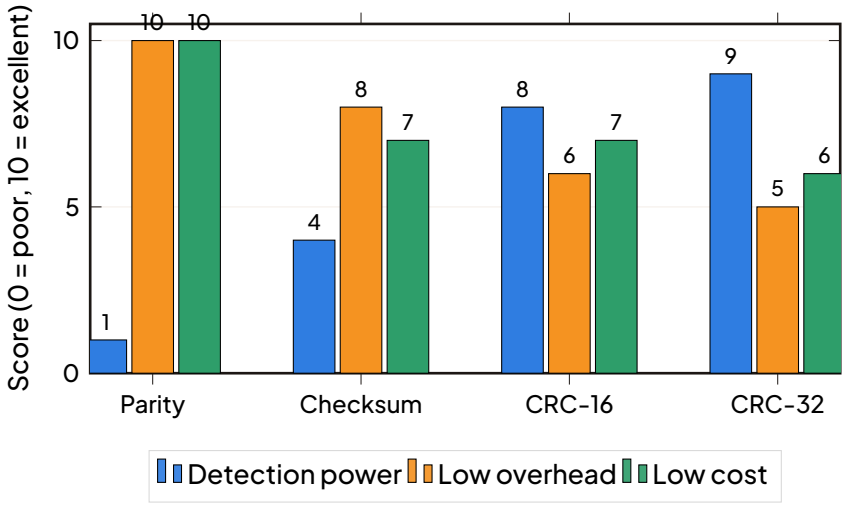
Name	Degree	Typical uses
CRC-8	8	Short frames, GSM/3G control channels, some sensor buses
CRC-16 (IBM)	16	USB packets, Modbus, XMODEM file transfer ($x^{16} + x^{15} + x^2 + 1$)
CRC-32 (Ethernet)	32	Ethernet frames (FCS field), ZIP archives, PNG images, Wi-Fi ($x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$)

DID YOU KNOW? **CRC around you in Cameroon**

Every Ethernet frame travelling through a CAMTEL or MTN backbone router ends with a 32-bit FCS computed with CRC-32; a corrupted frame is silently discarded and retransmitted by higher layers. When you unzip a downloaded document and the archiver reports a "CRC failed" message, it is precisely this remainder check that has detected a corrupted byte inside the archive.

3.3 Comparing Parity, Checksum and CRC

Criterion	Parity bit	Checksum	CRC
Redundancy (overhead)	1 bit	One word (8–32 bits)	n bits (8–32)
Single-bit error	Detected	Detected	Detected
Odd number of errors	Detected	Often detected	Detected if $(x+1)$ divides G
Double-bit error	Missed	Often detected	Detected
Burst of length $\leq n$	Missed	Not guaranteed	**All detected**
Reordered blocks	Missed	**Missed**	Detected
Compensating errors	Missed	**Missed**	Detected
Computation cost	Trivial (XOR tree)	Low (additions)	Moderate; very fast in hardware (shift registers + XOR gates)
Typical use	RAM, simple serial links	IP/TCP/UDP headers	Ethernet, USB, ZIP, storage



Qualitative comparison: detection power rises from parity to CRC-32, while overhead and computational cost increase moderately.

KEY POINTS **Key points**

- A **checksum** sums the data words; the one's complement version (IP/TCP) makes the receiver's total come out as all 1s.
- Checksums miss reordered blocks and compensating errors.
- A **CRC** divides the message polynomial by a generator $G(x)$ of degree n using XOR only (no borrows); the n-bit remainder is appended to the frame.
- Receiver: divide the whole frame by $G(x)$; remainder 0 ⇒ accepted.
- A degree-n CRC detects all bursts of length ≤ n and virtually all longer errors; CRC-32 protects Ethernet frames and ZIP files.
- The stronger the code, the higher the redundancy and cost: choose parity for trivial links, checksums for headers, CRC for frames and files.

EXERCISE 3 **Practice**

1. Four 16-bit words `2345`, `AB00`, `1F1F` and `70A6` (hexadecimal) are to be protected by a one's complement checksum. Compute the checksum, then verify it as the receiver would.
2. Using the generator $G(x) = x^3 + x + 1$ (bit string 1011), compute the CRC of the message

M = 1001101 and give the transmitted frame.

3. A receiver using the same G receives the frame 1001101 001. Perform the division and state whether the frame is accepted.
4. Explain why a checksum would fail to detect the corruption of a message in which two bytes have been exchanged, whereas a CRC would almost certainly detect it.

SOLUTION **Exercise 1**

1. Sum: `2345 + AB00 = CE45`; `+ 1F1F = ED64`; `+ 70A6 = 15E0A`. Fold the carry: `5E0A + 1 = 5E0B`. One's complement: `FFFF − 5E0B = A1F4`. Receiver: `5E0B + A1F4 = FFFF`: accepted.
2. Message M = 1001101 (7 bits). Append 3 zeros: 1001101000. Polynomial division by $G(x) = x^3 + x + 1$ (1011): $M(x) = x^6 + x^3 + x^2 + 1$. $x^3M(x) = x^9 + x^6 + x^5 + x^3$. Divide by $x^3 + x + 1$:
   - $x^6G = x^9 + x^7 + x^6$; rem $x^7 + x^5 + x^3$.
   - $x^4G = x^7 + x^5 + x^4$; rem $x^4 + x^3$.
   - $x^1G = x^4 + x^2 + x$; rem $x^3 + x^2 + x$.
   - $x^0G = x^3 + x + 1$; rem $x^2 + 1$.

   Remainder $R(x) = x^2 + 1 \rightarrow$ `101`. Transmitted frame: 1001101 101.
3. Received frame 1001101001. Divide by 1011: The correct remainder for the valid frame is 101. The received frame has 001 as the last 3 bits. Dividing 1001101001 by 1011 yields a non-zero remainder (specifically, the syndrome is non-zero). The frame is rejected; an error has been detected.
4. Addition is commutative: swapping two bytes leaves the sum unchanged, so the checksum still matches and the corruption is invisible. A CRC divides the bit sequence as a polynomial, where the position of every bit matters (it determines the power of x); swapping two bytes changes the dividend polynomial, hence (almost always) the remainder, so the receiver detects the error.

4 ERROR CORRECTION: HAMMING CODES AND APPLICATIONS

In the previous section we studied checksums and the Cyclic Redundancy Check (CRC). Both are excellent detection tools: they tell the receiver "something went wrong". But detection alone raises an awkward question: once you know a message is corrupted, what do you do? The usual answer is "ask for it again". In this section we study what happens when asking again is impossible, and we meet the great classic of the subject: the **Hamming code**, which not only detects an error but locates and corrects it automatically.

4.1 Why Correction Matters: When Retransmission Is Not an Option

Consider three concrete situations.

- **Broadcast and one-way links.** A television satellite over Yaoundé broadcasts the same signal to millions of receivers. If one receiver detects an error, it cannot ask the satellite to repeat just for itself: there is no return channel, or it would be absurdly expensive.
- **Data storage.** A file written on a disk or in memory today may be read back next year. By then, the "sender" (the disk head of last year) no longer exists. If a bit has flipped, nobody can retransmit: the data must heal itself.
- **Deep space and long delays.** A signal from a space probe takes minutes or hours to arrive. Retransmitting doubles an already enormous delay, and the probe may have moved on.

In all these cases we need **Forward Error Correction** (FEC): the sender adds more clever redundancy than a simple check, so that the receiver can **reconstruct** the original data even after some bits are damaged. The price is extra bits; the reward is autonomy. The simplest and most instructive FEC scheme is the family of Hamming codes, invented by Richard Hamming in 1950 after he grew frustrated with a computer that stopped every time it detected an error.

DID YOU KNOW? **Did you know?**

Richard Hamming worked at Bell Labs with relay computers that ran unattended over the weekend. The machine could detect an error using parity, but then it simply abandoned the whole job. Hamming reportedly said, in essence: "If the machine can detect an error, why can't it locate it and correct it?" The result was the family of codes that still protects your computer's memory today.

4.2 Hamming Distance: Measuring How Different Two Words Are

Before correcting errors, we need a ruler to measure "how far apart" two binary words are.

DEFINITION **Hamming distance**

The **Hamming distance** between two binary words of equal length is the number of positions in which they differ. It is computed by counting the 1s in the bitwise XOR of the two words. The minimum distance $d_{\min}$ of a code is the smallest Hamming distance between any two distinct valid codewords.

EXEMPLE **Computing Hamming distances**

Compare 1011001 and 1001011:

Position	1	2	3	4	5	6	7
Word A	1	0	1	1	0	0	1
Word B	1	0	0	1	0	1	1
Differ?	no	no	yes	no	no	yes	no

Two positions differ, so the Hamming distance is 2. Equivalently, $1011001 \oplus 1001011 = 0010010$, which contains two 1s.

Why does this matter? Imagine valid codewords as cities on a map, and errors as steps along the roads. If every pair of cities is at least d_{min} steps apart, then a small number of erroneous steps cannot turn one valid word into another valid one — the corruption is visible. And if the received word is closer to one city than to any other, we can guess which city it came from. This intuition is captured by a fundamental property.

PROPERTY **Detection and correction capability**

Let a code have minimum Hamming distance d_{min}. Then:

- it can detect up to d erroneous bits if and only if $d_{min} \geq d + 1$;
- it can correct up to d erroneous bits if and only if $d_{min} \geq 2d + 1$.

Justification (instructive, not required for the exam): to detect d errors, d flips must never transform a valid word into another valid word, so valid words must be more than d apart. To correct d errors, the corrupted word must remain closer to the original word than to any other valid word; two "correction balls" of radius d around distinct codewords must not overlap, which forces a separation of at least $2d + 1$.

EXEMPLE **Applying the property**

A simple parity bit gives $d_{min} = 2$: it detects 1 error ($2 = 1+1$) but corrects none ($2 < 2\times1+1 = 3$). Hamming(7,4) has $d_{min} = 3$: it corrects exactly 1 error ($3 = 2\times1+1$) and detects up to 2 errors ($3 = 2 + 1$) — but not both at once, a subtlety we return to below.

4.3 Structure of a Hamming(7,4) Codeword

Hamming's brilliant idea: instead of one parity bit covering everything, use several parity bits, each watching a different, overlapping group of positions. When an error occurs, the pattern of failed checks — the **syndrome** — points like a compass to the exact faulty position.

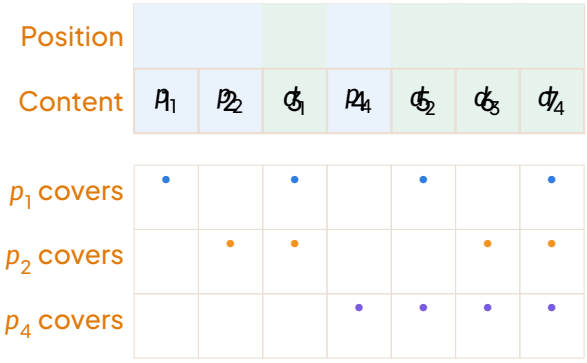
The construction rule is:

- Positions are numbered **from 1** (not 0).

- Positions that are **powers of 2** (1, 2, 4, 8, …) hold **parity bits**.
- All other positions (3, 5, 6, 7, 9, …) hold **data bits**, filled in order.
- Parity bit at position 2^k checks every position whose binary numeral has a 1 in bit k (with bit 0 being the least significant).

For Hamming(7,4): positions 1, 2, 4 are parity bits p_1, p_2, p_4; positions 3, 5, 6, 7 carry the four data bits $d_1 d_2 d_3 d_4$. The coverage groups are:

Parity bit	Checks positions	Binary reason
p_1 (pos. 1)	1, 3, 5, 7	numbers ending in 1 (001, 011, 101, 111)
p_2 (pos. 2)	2, 3, 6, 7	numbers with middle bit 1 (010, 011, 110, 111)
p_4 (pos. 4)	4, 5, 6, 7	numbers with leading bit 1 (100, 101, 110, 111)



Hamming(7,4) codeword: parity bits sit at positions 1, 2, 4 (powers of 2); each parity bit "covers" the positions whose binary index contains its bit.

WARNING **Number positions from 1, never from 0**

The single most common exam mistake is to number the positions 0, 1, 2, … The whole magic of the Hamming code rests on position numbers written in binary: parity bit p_1 watches positions whose binary index ends in 1, p_2 those with a 1 in the second binary digit, and so on. If you start numbering at 0, the powers-of-2 rule collapses, the syndrome no longer gives the error position, and every subsequent computation is wrong. Always write the position row 1 to 7 first.

4.4 Encoding a Hamming(7,4) Codeword

METHOD **Encoding a 4-bit dataword into Hamming(7,4) with even parity**

1. Write the position numbers 1 to 7. Place the four data bits, in order, at positions 3, 5, 6, 7.
2. For parity bit p_1 (position 1): count the 1s among positions 3, 5, 7. With even parity, set p_1 so the total over $\{1, 3, 5, 7\}$ is even.
3. For parity bit p_2 (position 2): do the same over positions $\{2, 3, 6, 7\}$.
4. For parity bit p_4 (position 4): do the same over positions $\{4, 5, 6, 7\}$.
5. Read the 7-bit codeword from position 1 to position 7.

EXEMPLE **Encoding the dataword 1011**

Data: $d_1 = 1$ (pos. 3), $d_2 = 0$ (pos. 5), $d_3 = 1$ (pos. 6), $d_4 = 1$ (pos. 7).

- p_1 over positions $\{3, 5, 7\} = \{1, 0, 1\}$: two 1s, already even $\Rightarrow p_1 = 0$.
- p_2 over positions $\{3, 6, 7\} = \{1, 1, 1\}$: three 1s, odd $\Rightarrow p_2 = 1$.
- p_4 over positions $\{5, 6, 7\} = \{0, 1, 1\}$: two 1s, even $\Rightarrow p_4 = 0$.

Codeword (positions 1–7): 0 1 1 0 0 1 1, i.e. 0110011.

EXPERIMENT **Spreadsheet simulation of Hamming(7,4) encoding**

Open a spreadsheet (LibreOffice Calc, Excel, or Google Sheets). In row 1, enter headers: `Pos` in A1, then `1` to `7` in B1:H1. In row 2, enter the role: `p1`, `p2`, `d1`, `p4`, `d2`, `d3`, `d4` in B2:H2. In row 3, enter your 4 data bits in D3, E3, F3, G3 (positions 3,5,6,7). In B3 (p1) enter formula: `=MOD(D3+E3+G3,2)`. In C3 (p2): `=MOD(D3+F3+G3,2)`. In E3 (p4): `=MOD(E3+F3+G3,2)`. The codeword appears in B3:H3. Test with data 1011 and verify you get 0110011. Then flip a bit in the codeword and build a decoder in row 5 using the same parity formulas on the received word to compute the syndrome.

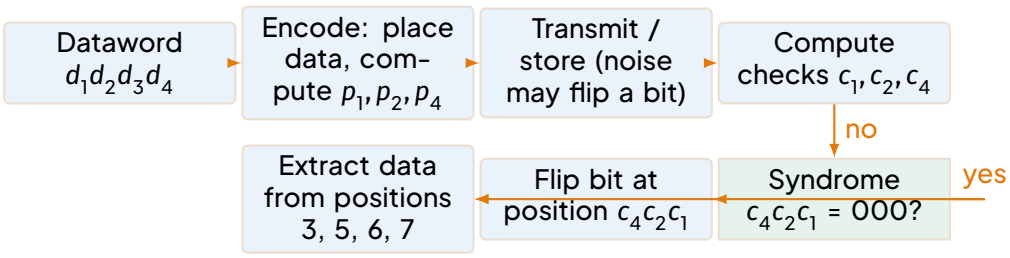
4.5 Decoding: Syndrome Computation and Correction

At the receiver, each parity group is re-checked. Each check produces one bit: 0 if the group has even parity (all fine), 1 if odd (something is wrong). Read in the order $c_4 c_2 c_1$, these bits form the **syndrome** — and here is Hamming's stroke of genius: **the syndrome, read as a binary number, is exactly the position of the erroneous bit**. A syndrome of 000 means "no error".

METHOD **Locating and correcting a single-bit error in Hamming(7,4)**

1. Recompute the parity check c_1 over positions $\{1, 3, 5, 7\}$: $c_1 = 1$ if the count of 1s is odd, else 0.
2. Similarly compute c_2 over $\{2, 3, 6, 7\}$ and c_4 over $\{4, 5, 6, 7\}$.
3. Form the binary number $c_4c_2c_1$. If it is 000, accept the word as correct.
4. Otherwise, convert $c_4c_2c_1$ to decimal: this is the position of the error. Flip that bit.
5. Extract the data bits from positions 3, 5, 6, 7.

Why does it work? A bit at position n belongs to exactly the parity groups corresponding to the 1s in the binary notation of n. If only that bit flips, exactly those checks fail — so the failed-check pattern spells out n in binary.



The Hamming encode–transmit–decode–correct pipeline. The syndrome decides: no error, or the exact position to flip.

WORKED EXAMPLE 5 **Complete worked example: encode, corrupt, correct**

A sender must transmit the dataword 1011 using Hamming(7,4) with even parity. During transmission, the bit at position 5 is flipped. Show how the receiver detects and corrects the error.

SOLUTION

Step 1 — Encode. As computed above, dataword 1011 gives the codeword

0110011 (positions 1–7).

Step 2 — Corrupt. Position 5 flips from 0 to 1. The receiver gets 0110111. Step 3 — Compute the syndrome.

- c_1 over positions $\{1, 3, 5, 7\} = \{0, 1, 1, 1\}$: three 1s, odd ⇒ $c_1 = 1$.
- c_2 over positions $\{2, 3, 6, 7\} = \{1, 1, 1, 1\}$: four 1s, even ⇒ $c_2 = 0$.

- c_4 over positions $\{4, 5, 6, 7\} = \{0, 1, 1, 1\}$: three 1s, odd $\Rightarrow c_4 = 1$.

Step 4 — Locate. Syndrome $c_4c_2c_1 = 101_2 = 5$. The error is at position 5. Step 5 — Correct. Flip position 5 back from 1 to 0: the word becomes 0110011 again. Step 6 — Extract. Data bits at positions 3, 5, 6, 7 give 1, 0, 1, 1: the original dataword 1011 is recovered, with no retransmission.

4.6 Limits of the Classic Hamming Code and the SECDED Extension

Hamming(7,4) has minimum distance 3: it is designed for exactly one erroneous bit per codeword. Two limitations follow.

WARNING **Hamming(7,4) mis-corrects double errors**

If two bits flip, the syndrome is non-zero, so the receiver believes there is a single error — but the syndrome now points to a third, innocent position. The receiver "corrects" the wrong bit and silently delivers bad data. For example, if positions 3 and 5 of our codeword 0110011 both flip, the received word is 0100111. The checks give $c_1 = 0$, $c_2 = 1$, $c_4 = 1$: syndrome $110_2 = 6$, so the receiver flips the healthy bit 6 and now the word carries three errors. Never claim in an exam that Hamming(7,4) "corrects errors" in general: it corrects single-bit errors only.

The standard fix is **SECDED** (Single Error Correction, Double Error Detection): add one extra overall parity bit covering the whole codeword (this is the Hamming(8,4) extended code). Now $d_{\text{min}} = 4$. Decoding rule:

- syndrome = 0 and overall parity OK: no error;
- syndrome ≠ 0 and overall parity fails: single error — correct it;
- syndrome = 0 and overall parity fails: the overall parity bit itself is faulty;
- syndrome ≠ 0 and overall parity OK: **double error** — detectable, but not correctable; request retransmission or flag the data.

SECDED is precisely the scheme used in **ECC memory** (Error-Correcting Code RAM), where typically 8 check bits protect each 64-bit word.

4.7 Applications and Chapter Summary

Error correction is everywhere in daily Cameroonian digital life, even when invisible:

- **ECC memory** in servers (including those hosting mobile-money platforms at MTN or Orange) uses SECDED Hamming codes so that a stray cosmic ray flipping a RAM bit does not corrupt an account balance.
- **QR codes** (used for payments, event tickets, exam result checking) use Reed–Solomon codes, a powerful relative of Hamming codes: this is why a QR code still scans even when partly

dirty or torn.

- **Barcodes** combine a check digit (detection, as in earlier lessons) with careful symbol design.
- **Mobile networks** (GSM, 4G) layer convolutional and block codes over CRC: the CRC detects, the FEC corrects, and retransmission (ARQ) is the last resort.
- **Deep-space and satellite links** rely almost entirely on forward error correction, for the reasons given at the start of this section.

KEY POINTS **Comparing the correctness methods of this chapter**

Method	Redundancy	Detects	Corrects
Parity bit	1 bit per word	any odd number of bit errors	nothing
Checksum	one sum per block	most burst errors (weak vs. compensating errors)	nothing
CRC	n bits per block	all bursts of length $\leq n$, most longer ones	nothing
Hamming(7,4)	3 bits per 4 data bits	up to 2 errors (or 1 + correction)	exactly 1 error
SECDED (Hamming(8,4))	4 bits per 4 data bits	1 error corrected and 2 errors detected	1 error

Key exam reflexes: number positions from 1; parity bits at powers of 2; the syndrome $c_4c_2c_1$ gives the error position in binary; detection needs $d_{min} \geq d + 1$, correction needs $d_{min} \geq 2d + 1$.

EXERCISE 4 **Practice**

1. Compute the Hamming distance between 1101011 and 1001111.
2. A code has minimum distance 5. How many errors can it detect? How many can it correct?
3. Encode the dataword 0110 using Hamming(7,4) with even parity.
4. A receiver gets the Hamming(7,4) word 1010101 (even parity). Compute the syndrome, decide whether there is an error, and if so correct it and give the dataword.

5. Explain why Hamming(7,4) cannot be trusted when two bits are corrupted, and how SECDED improves the situation.

SOLUTION **Exercise n°1**

1. $1101011 \oplus 1001111 = 0100100$: two 1s, so the distance is 2. 2. Detect: $d_{min} \geq d + 1 \Rightarrow d \leq 4$. Correct: $d_{min} \geq 2d + 1 \Rightarrow d \leq 2$. 3. Data at positions $3, 5, 6, 7$: $0, 1, 1, 0$. p_1 over $\{3, 5, 7\} = \{0, 1, 0\}$: one 1 $\Rightarrow p_1 = 1$. p_2 over $\{3, 6, 7\} = \{0, 1, 0\}$: one 1 $\Rightarrow p_2 = 1$. p_4 over $\{5, 6, 7\} = \{1, 1, 0\}$: two 1s $\Rightarrow p_4 = 0$. Codeword: 1100110. 4. c_1 over $\{1, 3, 5, 7\} = \{1, 1, 1, 1\}$: even, $c_1 = 0$. c_2 over $\{2, 3, 6, 7\} = \{0, 1, 0, 1\}$: even, $c_2 = 0$. c_4 over $\{4, 5, 6, 7\} = \{0, 1, 0, 1\}$: even, $c_4 = 0$. Syndrome 000: no error; dataword (positions $3, 5, 6, 7$) is 1101. 5. With two flipped bits the syndrome is non-zero and points to a wrong position, so the decoder flips a healthy bit and outputs corrupted data without any warning. SECDED adds an overall parity bit, raising d_{min} to 4: a double error then leaves the overall parity correct while the syndrome is non-zero, a combination the decoder recognises as an uncorrectable double error, which it flags instead of mis-correcting.

5 INTEGRATION ACTIVITY

6 INTEGRATION ACTIVITY: EXECUTING CORRECTNESS CHECKING METHODS

6.1 Aims of the activity

By the end of this integration activity, you should be able to:

- apply **even parity** to protect 7-bit ASCII characters and detect transmission errors;
- construct and exploit a **two-dimensional parity** (block parity) grid to locate and correct a single corrupted bit;
- compute a simple **checksum** for a block of data and use it to verify a second transmission;
- encode a 4-bit value with a **Hamming(7,4)** code, simulate a flipped bit and correct it using the syndrome;
- compare error-detection and error-correction techniques and **justify the choice** of a method for a given real-life context, in a clear written recommendation.

KEY POINTS **Skills mobilised**

This activity integrates the whole chapter: parity checking, block (2D) parity, checksums, CRC principles, Hamming codes, and the ability to argue a technical choice in writing — exactly the kind of structured reasoning expected at the GCE Advanced Level practical and theory papers.

6.2 Situation

EXPERIMENT **Case Study: Mr. Ngong's Cybercafé**

Mr. Ngong runs a small **cybercafé** near Commercial Avenue in Bamenda. Every year, after the Cameroon GCE Board releases results, candidates crowd into his café to download their result slips. The café's router is connected through an old telephone line that is noisy: electrical interference from a nearby welding workshop sometimes flips individual bits during download.

The results are transmitted as blocks of **7-bit ASCII codes**. Mr. Ngong has noticed that a downloaded slip occasionally displays a wrong character (for example a "C" becoming a "B" — a disaster when it is a grade!). He asks your class, as Upper Sixth ICT students, to

help him choose and apply a **correctness checking method**.
Your teacher provides the following working data.
Data set A — single characters. Each character is stored as 7 data bits, to which one even parity bit is appended as the 8th (most significant) bit. A received sample of four protected bytes is:

Byte received	7 data bits	Parity bit (MSB)
Byte 1	1000001	1
Byte 2	1000010	0
Byte 3	1000011	1
Byte 4	1000100	1

(Recall: ASCII 'A' = 1000001, 'B' = 1000010, 'C' = 1000011, 'D' = 1000100.)
Data set B — a block of 8 characters. The grades of one candidate are sent as the 8-character string AB2C1DEB (each character coded on 7 bits, even parity bit added per character). The block is then protected in addition by a two-dimensional parity scheme: the 8 bytes are written as 8 rows of 7 data bits; a row parity bit (even) is appended to each row, making 8 columns; then a column parity bit (even) is computed for each of the 8 columns and appended as a 9th row. The transmitted grid therefore has 9 rows and 8 columns (56 data bits + 16 parity bits). During download, exactly one bit of the grid is flipped by interference.
Data set C — checksum verification. The café's software treats each 7-bit character code as a decimal number and appends to the block a **checksum** equal to the sum of all the codes modulo 128 (so that it fits on 7 bits). For the block AB2C1DEB the codes are: A = 65, B = 66, 2 = 50, C = 67, 1 = 49, D = 68, E = 69, B = 66.
Data set D — a critical grade code. The overall grade of a candidate is coded on 4 bits $d_1 d_2 d_3 d_4$ (for example 0110). Because this field is critical, it is protected by a **Hamming(7,4)** code: the 4 data bits are placed in positions 3, 5, 6, 7 of a 7-bit word, and even parity bits p_1, p_2, p_4 are placed in positions 1, 2, 4, where p_1 covers positions 1, 3, 5, 7; p_2 covers positions 2, 3, 6, 7; p_4 covers positions 4, 5, 6, 7. During transmission of the grade 0110, one bit is flipped.
Mr. Ngong's question to the class: "Which method should I install permanently on my machines, and why?"

6.3 Tasks

1. **Parity check on single characters.** For each byte of data set A, verify the even parity. State which bytes contain an error and which are (apparently) correct. What characters should the correct bytes display?

2. **Limits of simple parity.** Suppose two bits of byte 1 had been flipped instead of one. Would the parity check still detect the error? Justify, and state the general limitation of single-bit parity.

3. **Two-dimensional parity.** Build the full parity grid for data set B (8 data rows, 7 data columns, row parity column, column parity row). Then suppose the received grid has a single flipped bit in row 3, column 5. Explain how the row and column parity failures locate the error, and correct it. What does this method offer that simple parity does not?
4. **Checksum.** Compute the checksum of data set C (sum modulo 128). A second transmission of the same block arrives with codes 65, 66, 50, 67, 49, 68, 69, **67** and the same checksum as before. Verify the checksum and conclude: is the second transmission accepted or rejected? Give one weakness of a simple sum checksum.
5. **Hamming(7,4).** Encode the grade code `0110` into a 7-bit Hamming word. Then flip the bit in position 6, compute the syndrome $c_4c_2c_1$ from the three parity checks, identify the erroneous position, and correct it. How many errors can Hamming(7,4) correct, and how many can it merely detect?
6. **Recommendation.** Write a short note (10–15 lines) to Mr. Ngong comparing the four techniques (cost in extra bits, detection power, correction power, ease of implementation) and recommending one method — or a combination — for his cybercafé, with justification.
7. **Plenary.** Each group presents its grid, its Hamming correction and its recommendation in 5 minutes; the class debates the best choice.

DEFINITION **Even Parity**

A parity bit is added to a set of data bits so that the total number of `1` bits (data + parity) is even. If the receiver counts an odd number of `1`s, an error is detected.

DEFINITION **Two-Dimensional Parity (Block Parity)**

Data bits are arranged in a rectangular array. A parity bit is computed for each row and each column (usually even parity). The row parity bits form an extra column; the column parity bits form an extra row. A single-bit error causes exactly one row and one column parity check to fail, locating the error at their intersection.

DEFINITION **Checksum**

A checksum is a value computed from a block of data (e.g., arithmetic sum modulo 2^n) and transmitted with the data. The receiver recomputes the checksum; a mismatch indicates corruption.

DEFINITION **Hamming(7,4) Code**

A single-error-correcting code that maps 4 data bits to 7 bits by inserting 3 parity bits

at positions that are powers of two (1, 2, 4). Each parity bit covers a specific subset of positions. The syndrome (recomputed parity checks) gives the binary position of a single flipped bit.

6.4 Model answer

Task 1 — Parity check on data set A.

Even parity means the total number of 1s (7 data bits + parity bit) must be even.

Byte	8-bit pattern (MSB first)	Number of 1s	Parity OK?
1	1 1000001	3 (odd)	**No** → Error detected
2	0 1000010	2 (even)	Yes → 'B' accepted
3	1 1000011	4 (even)	Yes → 'C' accepted
4	1 1000100	3 (odd)	**No** → Error detected

Bytes 1 and 4 are corrupted; bytes 2 and 3 are accepted and display 'B' and 'C'.

WARNING **Common mistake**

Do not forget to include the parity bit itself in the count. Even parity ⇔ total number of 1s is even.

Task 2 — Limit of simple parity.

If two bits of byte 1 flip, the number of 1s changes by -2, 0 or +2, so it remains even: the parity check sees nothing.

PROPERTY **Limitation of single-bit parity**

Single-bit even (or odd) parity detects any odd number of bit errors but fails for any even number of errors. It can never indicate which bit is wrong, hence no correction is possible.

Task 3 — Two-dimensional parity.

The 7-bit ASCII codes of `AB2C1DEB` are:

`A` = 65 = `1000001`
`B` = 66 = `1000010`
`2` = 50 = `0110010`
`C` = 67 = `1000011`
`1` = 49 = `0110001`
`D` = 68 = `1000100`
`E` = 69 = `1000101`
`B` = 66 = `1000010`

We write them as 8 rows of 7 data bits, compute even row parity (8th column), then even column parity for all 8 columns (9th row).

Char	Data bits (b1..b7)							Row parity
A	1	0	0	0	0	0	1	0
B	1	0	0	0	0	1	0	0
2	0	1	1	0	0	1	0	1
C	1	0	0	0	0	1	1	1
1	0	1	1	0	0	0	1	1
D	1	0	0	0	1	0	0	0
E	1	0	0	0	1	0	1	1
B	1	0	0	0	0	1	0	0
Col. parity	0	0	0	0	0	0	0	0

Verification of column parities: each column contains an even number of `1`s (0, 2, 4 or 6), so all column parity bits are `0`.

Now suppose the received grid has a flipped bit at **row 3, column 5** (the 5th data bit of character '2', originally `0`).

- Row 3 originally had three `1`s (bits 2,3,6) → row parity = `1`. After flip, row 3 has four `1`s → even parity, but the stored row parity bit is still `1` → **row 3 parity check fails**.
- Column 5 originally had two `1`s (rows D and E) → column parity = `0`. After flip, column 5 has three `1`s → odd parity → **column 5 parity check fails**.
- All other rows and columns still have even parity → their checks pass.

The intersection of the failing row (3) and failing column (5) pinpoints the corrupted bit. We flip it back (`1`→`0`), restoring '2' = `0110010`.

PROPERTY **Advantage of 2D parity over simple parity**

2D parity not only detects but also locates and corrects any single-bit error in the block,

at the cost of $m + n$ extra bits for an $m \times n$ data block (here $8 + 8 = 16$ extra bits for 56 data bits). It detects all 1-bit and most 2-bit errors, but cannot correct 2-bit errors.

Task 4 — Checksum.

Sum of original codes:

$$65 + 66 + 50 + 67 + 49 + 68 + 69 + 66 = 500.$$

Checksum = $500 \bmod 128 = 500 - 3 \times 128 = 500 - 384 = \boxed{116}$.

Second transmission codes: 65, 66, 50, 67, 49, 68, 69, **67** (last byte changed from 66 to 67). Sum = 501; $501 \bmod 128 = 501 - 384 = 117 \neq 116$.

The checksums differ, so the block is **rejected** and a re-download is triggered.

WARNING **Weakness of simple additive checksum**

A simple sum ignores the order of the bytes (permuting two characters gives the same sum) and compensating errors (one byte +1, another −1) go undetected. This is why real networks use **Cyclic Redundancy Check (CRC)**, based on polynomial division over GF(2), which detects burst errors and order changes much more reliably.

DID YOU KNOW? **CRC in Cameroon's networks**

CAMTEL and mobile operators (MTN Cameroon, Orange Cameroon) use CRC-32 (Ethernet, Wi-Fi) and CRC-16 (PPP, mobile air interface) on every frame. The **ANTIC** (Agence Nationale des Technologies de l'Information et de la Communication) recommends CRC for all government e-services to guarantee integrity of administrative data (tax returns, exam results, identity records).

Task 5 — Hamming(7,4).

METHOD **Hamming(7,4) encoding and decoding**

1. Number bit positions 1 to 7. Place parity bits at positions 1, 2, 4 (powers of two). Place data bits d_1, d_2, d_3, d_4 at positions 3, 5, 6, 7.
2. Compute even parity bits: p_1 covers positions 1,3,5,7; p_2 covers positions 2,3,6,7; p_4 covers positions 4,5,6,7.
3. Transmit the 7-bit word.
4. On reception, recompute the three parity checks c_1, c_2, c_4 (1 if odd, 0 if even).

5. Syndrome $S = c_4c_2c_1$ (binary) gives the position of the error (0 = no error).
6. Flip the bit at position S to correct.

Encoding 0110: $d_1 = 0$ (pos 3), $d_2 = 1$ (pos 5), $d_3 = 1$ (pos 6), $d_4 = 0$ (pos 7).

$$p_1 : \text{positions } 3,5,7 \rightarrow 0+1+0 = 1 \Rightarrow p_1 = 1$$
$$p_2 : \text{positions } 3,6,7 \rightarrow 0+1+0 = 1 \Rightarrow p_2 = 1$$
$$p_4 : \text{positions } 5,6,7 \rightarrow 1+1+0 = 2 \Rightarrow p_4 = 0$$

Transmitted word (positions 1–7): `1100110`.

Flip position 6: received `1100100`.

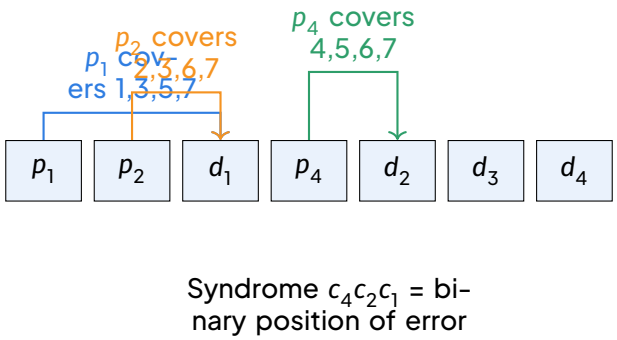
Syndrome computation:

$$c_1 : \text{pos } 1,3,5,7 = 1+0+1+0 = 2\,(\text{even}) \Rightarrow c_1 = 0$$
$$c_2 : \text{pos } 2,3,6,7 = 1+0+0+0 = 1\,(\text{odd}) \Rightarrow c_2 = 1$$
$$c_4 : \text{pos } 4,5,6,7 = 0+1+0+0 = 1\,(\text{odd}) \Rightarrow c_4 = 1$$

Syndrome $c_4c_2c_1 = 110_2 = 6$: error at **position 6**. Flip it back → `1100110`, data bits = `0110` — grade recovered.

PROPERTY **Hamming(7,4) capabilities**

- Corrects any single-bit error (1-bit error correction).
- Detects (but cannot correct) all double-bit errors (2-bit error detection).
- Cannot handle 3 or more errors reliably.



Hamming(7,4) bit positions and parity coverage

Task 6 — Recommendation (model note).

To Mr. Ngong.

We tested four methods on your downloads.

- **Simple parity** (1 extra bit per character) is very cheap and catches single-bit errors, but misses any even number of errors and cannot repair anything.
- **Two-dimensional parity** (16 extra bits per 8-character block) detects and corrects a single corrupted bit — ideal for small, fixed-size blocks like result slips when re-download is impossible.
- **Checksums** (7 extra bits per block) are extremely cheap and catch most random corruptions, but can be fooled by swapped or compensating errors; for stronger protection, a **CRC** (as used by CAMTEL/MTN/Orange on every frame) is the standard.
- **Hamming(7,4)** (3 extra bits per 4 data bits, 75% overhead) automatically corrects single errors and is perfect for tiny critical fields such as the grade code.

Recommendation: Deploy a **CRC (or at minimum a checksum)** on every downloaded block to detect corruption and trigger automatic re-download; use **Hamming coding** on the critical grade field for on-the-fly correction; keep **2D parity** as a manual recovery tool when the line is too noisy for re-download. This combination gives both detection and correction at a modest cost in extra bits and fits the GCE syllabus requirements for robust data transfer.

WARNING **Common mistakes to avoid**

- Do not forget to count the parity bit itself when checking even parity.
- In Hamming codes, number positions from 1 (not 0) and read the syndrome as $c_4c_2c_1$ (most significant parity check first).
- A checksum that matches does not prove the data are correct — it only makes undetected errors unlikely.
- In 2D parity, the column parity row must be computed after the row parity column has been added.

7 METHODS AND WORKED EXAMPLES

8 ICT11: EXECUTING DIFFERENT CORRECTNESS CHECKING METHODS

8.1 Errors in Data Transmission and the Need for Checking

DID YOU KNOW? **Why checking matters in Cameroon**

Every time you send money via Mobile Money (MTN MoMo, Orange Money), browse a government portal like e-Gouvernance, or stream a match on Canal+ via fibre, bits travel through CAMTEL's backbone, satellite links, or microwave relays. Noise, crosstalk, attenuation, and interference can flip bits. Without error detection and correction, a single flipped bit could change a transfer amount, corrupt a database record, or crash a video frame. This chapter teaches the methods that keep digital communication reliable — from simple parity to the CRC that protects every Ethernet frame and Wi-Fi packet.

DEFINITION **Error, Error Detection, Error Correction**

An **error** is any unwanted change in a bit ($0 \leftrightarrow 1$) during transmission or storage. **Error detection** allows the receiver to know that the received data is corrupted. **Error correction** enables the receiver to locate and fix the corrupted bit(s) without retransmission.

KEY POINTS **Error models**

- Single-bit error: only one bit in a frame is flipped (common in optical fibre, good copper).
- Burst error: a contiguous sequence of bits is corrupted (common in wireless, lightning strikes, crosstalk).
- Random errors: independent bit flips scattered across the frame.

The choice of checking method depends on the dominant error model and whether retransmission is feasible (ARQ) or not (forward error correction, FEC).

8.2 Skill 1: Applying Single-Bit Parity Checking

METHOD **Detecting an error with a parity bit**

1. Identify the **parity scheme**: even parity (total number of 1s, parity bit included, must be even) or odd parity (total number of 1s must be odd).
2. To generate the parity bit: count the 1s in the data word. For even parity, append 0 if the count is already even, 1 if it is odd. For odd parity, do the opposite.
3. To check a received word: count all the 1s (data + parity bit). If the total matches the scheme (even/odd), the word is accepted; otherwise an error is detected and re-transmission is requested.
4. Remember the limit: a single parity bit detects any **odd number of bit errors** (1, 3, 5…) but fails to detect an **even number of errors** (2, 4, …).

Reflex: parity bit = XOR of all data bits (even parity). If you can compute an XOR, you can compute parity.

WARNING **Parity bit position**

The parity bit can be placed at the MSB (most significant bit) or LSB (least significant bit). The GCE exam usually states "append a parity bit" (LSB) or "prefix". Always read the question carefully. The detection logic is identical regardless of position.

WORKED EXAMPLE 6 **Parity bit generation and checking**

A computer in a cybercafé in Buea sends the 7-bit ASCII code 1000001 (letter 'A') using even parity.

1. Determine the parity bit and the full 8-bit transmitted word.
2. The receiver receives 1001001 (the 4th bit was flipped). Does the receiver detect the error? Justify.
3. The receiver receives 11101001 (two bits flipped). Does the receiver detect the error? Comment.

SOLUTION

1. Generating the parity bit.
Data word: 1 0 0 0 0 0 1. Number of 1s = 2, which is already even. For even parity, the parity bit must keep the total even, so $P = 0$.

Transmitted word (data + parity bit appended at the end): 10000010.

2. One bit flipped.
Received word: 10010010. Count the 1s: positions 1, 4 and 7 give 3 ones → odd total. Even parity was expected, but the count is odd ⇒ the receiver detects the error and asks for retransmission. Note that the receiver knows that an error occurred, but not where.

3. Two bits flipped.
Received word: 11010010. Number of 1s = 4 → even. The parity test passes: the error is not detected, and the corrupted byte is accepted as if it were correct.

Conclusion. A single parity bit is cheap (only 1 extra bit per word) and detects all single-bit errors, which are the most frequent on good lines; but it is blind to double errors. This is why more powerful methods (checksums, CRC) are used in real networks such as CAMTEL's fibre links.

8.3 Skill 2: Using Two-Dimensional (Block) Parity

METHOD **Locating an error with block parity**

1. Arrange the data as a **matrix** of rows (bytes/words).
2. Compute a row parity bit for each row and a column parity bit for each column, using the agreed scheme (usually even parity).
3. At reception, recompute all parities. If exactly one row parity and one column parity fail, the error is at their intersection: flip that bit to correct it.
4. If only row parities (or only column parities) fail, or several of each fail, there are multiple errors: request retransmission.

Key idea: 2D parity not only detects but can correct a single-bit error, because the faulty row and faulty column "cross" at the bad bit.

KEY POINTS **2D parity capabilities**

- Detects all single-bit errors.
- Corrects all single-bit errors (unique intersection).
- Detects most double-bit errors (but cannot correct them).
- Fails to detect some patterns of 4 or more errors forming a rectangle.

Overhead: $(m + 1)(n + 1)$ bits for $m \times n$ data bits.

WORKED EXAMPLE 7 **Correcting a bit with 2D even parity**

A school server sends four 7-bit words protected by even row and column parity. The receiver obtains the following block (last column = row parities, last row = column parities):

Data bits								Row par.
1	0	1	1	0	1	0	0	0
0	1	1	0	1	0	1	0	0
1	1	0	1	0	0	1	1	1
0	0	1	0	1	1	0	1	0
0	0	1	0	0	0	0	0	

Last row: column parities

Check the block, locate any single-bit error and correct it.

SOLUTION

Step 1: check row parities (even parity expected).
Row 1: 1 + 0 + 1 + 1 + 0 + 1 + 0 + 0 = 4 ones, parity bit 0 → total 4, even. OK.
Row 2: 0 + 1 + 1 + 0 + 1 + 0 + 1 + 0 = 4 ones, parity 0 → even. OK.
Row 3: 1 + 1 + 0 + 1 + 0 + 0 + 1 + 1 = 5 ones, parity 1 → total 6, even. OK.
Row 4: 0 + 0 + 1 + 0 + 1 + 1 + 0 + 1 = 4 ones, parity 0 → even. OK.

Step 2: check column parities.
Col 1: 1 + 0 + 1 + 0 = 2, parity 0 → even. OK. Col 2: 0 + 1 + 1 + 0 = 2, parity 0 → OK. Col 3: 1 + 1 + 0 + 1 = 3, parity 1 → total 4, OK.
Col 4: 1 + 0 + 1 + 0 = 2, parity 0 → OK. Col 5: 0 + 1 + 0 + 1 = 2, parity 0 → OK. Col 6: 1 + 0 + 0 + 1 = 2, parity 0 → OK.
Col 7: 0 + 1 + 1 + 0 = 2, parity 0 → OK. Col 8: 0 + 0 + 1 + 1 = 2, parity 0 → OK.

All checks pass: the block contains no detectable error.

Variant (the typical GCE question). Suppose instead that row 2's parity had failed and column 5's parity had failed. The error would lie at the intersection: row 2, column 5. The received bit there is 1; we flip it to 0. Row 2 then contains 0, 1, 1, 0, 0, 0, 1, 0 = 3 ones, so its parity bit must also be corrected to 1 (or the parity bits are simply recomputed after correction). The corrected data word of row 2 is 0110001.
Lesson: one failing row + one failing column ⇒ correctable single error; any other pattern ⇒ retransmission.

8.4 Skill 3: Computing and Verifying a Checksum

METHOD **Checksum by addition (sender and receiver)**

1. Split the message into blocks of fixed size (e.g. bytes or 16-bit words).
2. Sender: add all blocks (ordinary or 1's-complement addition). Take the complement (or truncate the carry, depending on the convention): this is the **checksum**, appended to the message.
3. Receiver: add all blocks including the checksum. With the complement convention, the result must be all 1s (e.g. 11111111); with the truncation convention, the sum of data + checksum must give the expected residue (often 0 modulo 2^n).
4. If the test fails ⇒ error detected, retransmission requested. If it passes ⇒ data accepted (but not guaranteed: compensating errors can cancel out).

Reflex: the checksum is designed so that "data + checksum" gives a constant, known result at the receiver.

WARNING **Checksum conventions**

Two common conventions exist:

- Modulo 2^n (truncation): sum all blocks, discard overflow beyond n bits; checksum = $(2^n - \text{sum}) \bmod 2^n$. Receiver checks if total sum $\equiv 0 \pmod{2^n}$.
- 1's complement (Internet checksum): add with end-around carry; checksum = 1's complement of sum. Receiver adds all including checksum; result must be all 1s.

The GCE syllabus typically uses the modulo 2^n (byte or word) convention. Always state which convention you are using.

WORKED EXAMPLE 8 **Byte checksum modulo 256**

A mobile money application sends the amount code as three bytes: 45, 120, 78 (decimal). The checksum is defined by: the sum of the three data bytes and the checksum byte must be a multiple of 256.

1. Compute the checksum byte.
2. The receiver gets 45, 120, 78, 13. Verify.
3. The receiver gets 45, 121, 78, 13. What happens?

SOLUTION

1. Computing the checksum.
Sum of data: 45 + 120 + 78 = 243.
We need $243 + C \equiv 0 \pmod{256}$, so $C = 256 - 243 = \boxed{13}$.

The sender transmits: 45, 120, 78, 13.

2. Verification of a correct frame.
45 + 120 + 78 + 13 = 256 = 1 × 256. The sum is a multiple of 256 ⇒ the frame is accepted.

3. Verification of a corrupted frame.
45+121+78+13 = 257, which is not a multiple of 256 ⇒ error detected; the frame is rejected and retransmission is requested.

Remark (limit of the method). If two bytes are corrupted in opposite directions (e.g. 45 → 46 and 120 → 119), the sum is unchanged and the checksum does not detect anything. This is why the CRC below is preferred in real protocols.

8.5 Skill 4: Computing a CRC by Polynomial Division

DEFINITION **Generator polynomial**

A **generator polynomial** $G(x)$ of degree r is a binary polynomial (coefficients 0 or 1) with the highest and lowest terms equal to 1. It is represented as an $(r + 1)$-bit divisor. The CRC will be r bits long.

METHOD **Cyclic Redundancy Check (CRC)**

1. Write the generator polynomial $G(x)$ as a binary divisor (e.g. $x^3+1 \rightarrow 1001$). If the divisor has $r + 1$ bits, the CRC has r bits.
2. Sender: append r zeros to the data word M.
3. Divide (in binary, using **XOR** instead of subtraction — no borrows, no carries) the extended word by the divisor.
4. The remainder (r bits) is the CRC. Transmitted frame = original data + CRC.
5. Receiver: divide the whole received frame by the same divisor. Remainder = 0 ⇒ accepted; remainder ≠ 0 ⇒ error detected.

Reflexes: XOR rule: $1 \oplus 1 = 0, 1 \oplus 0 = 1, 0 \oplus 0 = 0$. The remainder must always be exactly r bits (pad with leading zeros if needed).

KEY POINTS **CRC error detection guarantees**

For a generator polynomial of degree r (CRC length r):

- Detects all single-bit errors.
- Detects all double-bit errors if $G(x)$ has a factor with at least three terms.
- Detects all odd numbers of errors if $G(x)$ contains $(x + 1)$ as a factor.
- Detects all burst errors of length $\leq r$.
- Detects most burst errors of length $r + 1$ (probability $1 - 2^{-r}$).
- Detects most longer bursts (probability $1 - 2^{-r}$).

Standard polynomials: CRC-8 ($x^8 + x^2 + x + 1$), CRC-16 ($x^{16} + x^{15} + x^2 + 1$), CRC-32 (Ethernet, ZIP, PNG).

WORKED EXAMPLE 9 **CRC with generator $x^3 + x + 1$**

The data to transmit is $M = 110101$ and the generator is $G(x) = x^3 + x + 1$.

1. Give the binary divisor and the number of CRC bits.
2. Compute the CRC and the transmitted frame.
3. Show what the receiver computes when the frame arrives intact.

SOLUTION

1. Divisor and CRC length.
$G(x) = x^3 + x + 1 = 1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 1 \rightarrow$ divisor 1011 (4 bits) $\Rightarrow r = 3$ CRC bits.

2. Computing the CRC.
Append $r = 3$ zeros: 110101 000. Divide by 1011 using XOR:

```
110101000
⊕ 1011    (aligned on the leading 1)
-----------
 011001000
  ⊕ 1011
 ----------
  010101000
   ⊕ 1011
  ----------
   001100000
    ⊕ 1011
   ----------
    000111000
     ⊕ 1011
    ----------
     000010100
      ⊕ 1011
     ----------
      000001110
       ⊕ 1011
      ----------
       0000101
```

The final remainder is 101 (3 bits): this is the CRC.
Transmitted frame = data + CRC = 110101101.

3. Verification at the receiver.
The receiver divides 110101101 by 1011 with the same XOR procedure. Since the transmitted frame was built as (data shifted) ⊕ remainder, the division gives remainder 000 ⇒ the frame is accepted.
If any bit had been flipped during transmission (e.g. on a noisy radio link between a village and Yaoundé), the remainder would be non-zero and the error would be detected. CRC detects all burst errors of length ≤ r and the vast majority of longer bursts, which is why it is used in Ethernet, Wi-Fi and USB.

8.6 Skill 5: Building a Hamming Code

METHOD **Encoding with Hamming code**

1. Parity positions: place parity bits at positions 1, 2, 4, 8, … (powers of 2). Data bits fill the other positions 3, 5, 6, 7, …
2. How many parity bits? For m data bits, choose k such that $2^k \geq m + k + 1$. For $m = 4$, $k = 3$ (Hamming(7,4)).
3. Coverage: parity bit at position 2^j checks all positions whose binary index has a 1 in bit

j (LSB = bit 0):

- P_1 (pos. 1) checks positions 1, 3, 5, 7 (binary indices with bit 0 = 1);
- P_2 (pos. 2) checks positions 2, 3, 6, 7 (bit 1 = 1);
- P_4 (pos. 4) checks positions 4, 5, 6, 7 (bit 2 = 1).

4. Set each parity bit for even parity over its group.

Key idea: each data bit is covered by a unique combination of parity bits, so the failing parity bits will "point" to the erroneous position.

WARNING **Syndrome bit order**

When forming the syndrome $C = (P_4\,P_2\,P_1)_2$, the highest-position parity bit is the most significant bit. Reversing the order gives the wrong position. Always write the syndrome with $P_{2^{k-1}}$ as MSB down to P_1 as LSB.

WORKED EXAMPLE 10 **Encoding the nibble 1011 in Hamming(7,4)**

Encode the 4 data bits $d_1d_2d_3d_4 = 1011$ using the Hamming(7,4) code with even parity. Give the 7-bit codeword.

SOLUTION

Step 1: place the bits.
Positions 1, 2, 4 are parity bits P_1, P_2, P_4. Data bits go to positions 3, 5, 6, 7:

Position	1	2	3	4	5	6	7
Content	P_1	P_2	d_1=1	P_4	d_2=0	d_3=1	d_4=1

Step 2: compute P_1 (positions 1, 3, 5, 7): data bits involved: pos. 3 = 1, pos. 5 = 0, pos. 7 = 1 → two 1s, already even ⇒ $P_1 = 0$.
Step 3: compute P_2 (positions 2, 3, 6, 7): pos. 3 = 1, pos. 6 = 1, pos. 7 = 1 → three 1s, odd ⇒ $P_2 = 1$.
Step 4: compute P_4 (positions 4, 5, 6, 7): pos. 5 = 0, pos. 6 = 1, pos. 7 = 1 → two 1s, even ⇒ $P_4 = 0$.

Codeword (positions 1 to 7): $\boxed{0110011}$.

Check: group of P_1: 0, 1, 0, 1 → 2 ones, even ✓; group of P_2: 1, 1, 1, 1 → 4 ones, even ✓; group of P_4: 0, 0, 1, 1 → 2 ones, even ✓.

8.7 Skill 6: Detecting and Correcting an Error with Hamming Code

METHOD **Hamming syndrome decoding**

1. On reception, recompute each parity group (P_1, P_2, P_4, …) over the received word.
2. For each group, write 0 if parity is satisfied, 1 if it fails.
3. Read the results as a binary number $C = (P_4\,P_2\,P_1)_2$ (most significant bit = highest-position parity): this is the **syndrome**.
4. Syndrome = 0 ⇒ no error detected. Syndrome = $C \neq 0$ ⇒ the bit at position C is wrong: flip it to correct.
5. If more than one bit is wrong, Hamming(7,4) may miscorrect: it corrects exactly one error per codeword.

KEY POINTS **Hamming(7,4) properties**

- Codeword length: 7 bits (4 data + 3 parity).
- Minimum Hamming distance: 3.
- Can correct all single-bit errors.
- Can detect all double-bit errors (but cannot correct them; if you attempt correction, you will miscorrect).
- Overhead: 3/7 ≈ 43% redundancy. For larger blocks, extended Hamming codes (SECDED) add an overall parity bit to detect double errors.

WORKED EXAMPLE 11 **Correcting a received Hamming word**

The codeword 0110011 from the previous example is sent. Due to interference on the line, the receiver gets 0100011 (position 3 flipped). Apply Hamming decoding to detect and correct the error.

SOLUTION

Received word, positions 1 to 7: 0, 1, 0, 0, 0, 1, 1.

Step 1: check group P_1 (positions 1, 3, 5, 7): 0, 0, 0, 1 → one 1, odd ⇒ fails ⇒ bit = 1.
Step 2: check group P_2 (positions 2, 3, 6, 7): 1, 0, 1, 1 → three 1s, odd ⇒ fails ⇒ bit = 1.
Step 3: check group P_4 (positions 4, 5, 6, 7): 0, 0, 1, 1 → two 1s, even ⇒ OK ⇒ bit = 0.

Step 4: syndrome. $C = (P_4\, P_2\, P_1)_2 = (0\,1\,1)_2 = 3$.
The error is at position 3. Flip it: $0 \to 1$.
Corrected word: $0\,1\,1\,0\,0\,1\,1$, i.e. exactly the transmitted codeword, and the recovered data (positions 3, 5, 6, 7) is $\boxed{1011}$.

Why it works: position 3 is written 011 in binary, so it is covered by P_1 and P_2 but not P_4 — precisely the pattern of failing checks. The syndrome literally is the address of the error.

8.8 Skill 7: Choosing the Appropriate Checking Method

METHOD **Selecting a correctness-checking technique**

1. Need only detection, very cheap, low error rate (e.g. memory inside a machine, simple serial links): single **parity bit**.
2. Detection + correction of isolated bit errors (RAM memory, satellite links where retransmission is costly): **Hamming code** or 2D parity.
3. Detection of burst errors in frames (Ethernet, Wi-Fi, USB, mobile networks): **CRC**.
4. Simple software check of a file or record (quick integrity test, e.g. verifying a downloaded file): **checksum**.
5. Always compare the cost (number of redundant bits, computation time) with the error model of the channel (isolated bits vs bursts) and the possibility of retransmission.

DID YOU KNOW? **Real-world usage in Cameroon**

- Parity bit: UART serial ports (old modems, some POS terminals), RAM parity (older PCs).
- 2D parity: RAID-5/6 storage arrays (parity across disks), some legacy synchronous protocols.
- Checksum: IP header checksum, UDP/TCP pseudo-header, file integrity (MD5/SHA are cryptographic hashes, not simple checksums).
- CRC: Ethernet (CRC-32), Wi-Fi (CRC-32/16), USB (CRC-16/5), Bluetooth, MTN/Orange mobile money USSD frames, CAMTEL fibre OTN frames.
- Hamming/SECDED: ECC RAM in servers (banks, ministries), satellite comms (e.g. CAMSAT links), QR codes (Reed-Solomon, a generalization).

WORKED EXAMPLE 12 **Comparing methods in a GCE-style scenario**

An ISP in Douala transmits customer records of 1000 bits each over a fibre link where errors occur in bursts of up to 8 consecutive bits. Retransmission is possible and cheap. The company hesitates between: (a) a single parity bit, (b) Hamming code, (c) a 16-bit CRC. Advise the company, justifying with the properties of each method.

SOLUTION

Analyse the error model: bursts of up to 8 bits ⇒ several bits of the same frame are corrupted together.

(a) Single parity bit: it detects only an odd number of erroneous bits. A burst of 8 bits flips an even number of bits (if all 8 flip, parity is unchanged) ⇒ the error can pass unnoticed. Rejected.

(b) Hamming code: it corrects exactly one error per codeword and misbehaves with multiple errors. A burst of 8 bits inside one codeword would either be miscorrected or undetected. Moreover, Hamming adds about k bits per $2^k - k - 1$ data bits, and here correction is useless because retransmission is cheap. Rejected.

(c) 16-bit CRC: a CRC with $r = 16$ bits detects all burst errors of length ≤ 16 (hence all bursts of length 8), all odd numbers of bit errors, and the overwhelming majority of longer bursts. Cost: only 16 extra bits per 1000-bit frame, i.e. an overhead of $16/1000 = 1.6\%$, and detection is done by a fast hardware XOR division. Since retransmission is cheap, detection (not correction) is sufficient.

Conclusion: the company should choose the 16-bit CRC: it matches the burst-error model, detects all bursts of length up to 16, costs little bandwidth, and combines naturally with a retransmission protocol (ARQ). This is precisely the strategy used in real frame-based networks such as Ethernet.

EXERCISE 5 **Practice exercises for GCE preparation**

1. A device uses odd parity. The data byte is 1011010. What is the transmitted byte (parity bit appended)?
2. A 2D even parity block of 3 rows × 4 data columns arrives with row parities [0, 1, 0] and column parities [1, 0, 1, 1]. Row 2 and column 3 fail. Locate and correct the error.
3. Compute the Internet checksum (1's complement, 16-bit) for the three 16-bit words: $0x1234$, $0x5678$, $0x9ABC$. Show the steps including end-around carry.
4. For generator $G(x) = x^4 + x + 1$ (divisor 10011), compute the CRC for data 101001. Give the transmitted frame.
5. Encode the 8-bit value 11010011 using Hamming code. How many parity bits are needed? Give the full codeword.

6. A Hamming(7,4) codeword 1011010 is received. Compute the syndrome and correct if necessary. What are the original data bits?

SOLUTION **Exercise 1**

Odd parity: data 1011010 has four 1s (even). To make total odd, parity bit = 1. Transmitted: 10110101.

SOLUTION **Exercise 2**

Row 2, column 3 intersection. Flip that data bit. Recompute affected row/column parities.

SOLUTION **Exercise 3**

$0x1234 + 0x5678 = 0x68AC$; $+0x9ABC = 0x10368$; end-around carry: $0x0368 + 0x1 = 0x0369$; 1's complement = $0xFC96$. Checksum = $0xFC96$.

SOLUTION **Exercise 4**

Divisor 10011 ($r = 4$). Append 4 zeros: 1010010000. XOR division yields remainder 0110. Frame: 1010010110.

SOLUTION **Exercise 5**

$m = 8$, need k such that $2^k \geq 8 + k + 1$. $k = 4$ ($16 \geq 13$). Hamming(12,8). Parity positions 1,2,4,8. Compute each parity over its group. Codeword length 12.

SOLUTION **Exercise 6**

Received 1011010. Check P_1 (1.3,5.7): $1, 1, 0, 0 = 2$ even OK (0). P_2 (2.3,6.7): $0, 1, 1, 0 = 2$ even OK (0). P_4 (4.5,6.7): $1, 0, 1, 0 = 2$ even OK (0). Syndrome 000 $\Rightarrow$ no error. Data bits (3.5,6.7) = 1010.

9 EXERCISES

9.1 I check my knowledge

EXERCISE 6 **Multiple choice questions**

For each question, choose the single correct answer.

1. A single parity bit added to a byte allows the receiver to:
   (a) correct any single-bit error
   (b) detect any single-bit error
   (c) detect any double-bit error
   (d) correct any double-bit error
2. In even parity, the parity bit is chosen so that:
   (a) the number of 1s in the codeword is odd
   (b) the number of 0s in the codeword is even
   (c) the number of 1s in the codeword is even
   (d) the codeword ends with a 0
3. The Cyclic Redundancy Check (CRC) is based on:
   (a) addition of all data bytes
   (b) polynomial division in modulo-2 arithmetic
   (c) counting the number of 1s
   (d) interleaving of data bits
4. A Hamming(7,4) code contains:
   (a) 7 data bits and 4 control bits
   (b) 4 data bits and 3 control bits
   (c) 7 data bits and 3 control bits
   (d) 4 data bits and 4 control bits

EXERCISE 7 **True or false — justify**

State whether each assertion is true or false, and justify your answer in one or two sentences.

1. "A single parity bit detects all transmission errors."
2. "A simple additive checksum detects the swapping of two data blocks."
3. "A CRC can detect all burst errors shorter than the degree of the generator polynomial."
4. "A Hamming(7,4) code can correct one erroneous bit per codeword."

EXERCISE 8 **Definitions**

Define each of the following terms in one or two clear sentences, as expected at the GCE Advanced Level:

1. **parity bit**
2. **checksum**
3. **Hamming distance** between two codewords
4. **syndrome** (in a Hamming code)

EXERCISE 9 **Direct calculations**

1. The 7-bit ASCII code of the letter 'K' is 1001011. Compute the parity bit to append for even parity, then for odd parity.
2. Compute the Hamming distance between the codewords 1011010 and 1001110.
3. A code has minimum Hamming distance $d_{min} = 3$. How many bit errors can it detect? How many can it correct?

9.2 I practise

EXERCISE 10 **Parity check on received bytes**

A system transmits characters using 7-bit ASCII with even parity, the parity bit being placed as the most significant bit (bit 8). The following bytes are received:

Byte received	Content
B_1	11001011
B_2	01010010
B_3	11100001

1. For each byte, state whether it is valid under even parity, showing your counting.
2. For each invalid byte, explain why the parity method cannot tell which bit is wrong.
3. Give one situation in which a corrupted byte would still be declared valid by even parity.

EXERCISE 11 **Two-dimensional parity**

A block of 4 rows of 7 data bits is protected by even parity on each row and each column (the bottom-right bit checks both its row and its column). The received block is:

Row	Data bits							Row parity
1	1	0	1	1	0	0	1	0
2	0	1	1	0	1	0	0	1
3	1	1	0	1	1	0	1	1
4	0	0	1	0	0	1	1	1
Column parity	0	0	1	0	0	1	1	

Exactly one bit of the block is wrong.

1. Identify the row and the column of the erroneous bit, explaining your reasoning.
2. Give the corrected value of that bit.
3. Explain why two-dimensional parity can correct a single-bit error whereas simple parity cannot.

EXERCISE 12 **Additive checksum**

A device sends four data bytes followed by an 8-bit additive checksum (the sum of the bytes modulo 256, written on 8 bits):

$$B_1 = 01100110, \quad B_2 = 10101010, \quad B_3 = 00011110, \quad B_4 = 11000011.$$

1. Compute the checksum byte transmitted by the sender.
2. The receiver gets B_1, B_2, B_3, B_4 and the checksum, but B_2 has arrived corrupted as

10101000. Show the computation performed by the receiver and state whether the error is detected.

3. Give an example of a corruption of two bytes that this checksum would not detect.

EXERCISE 13 **CRC computation**

A link uses the generator polynomial $G(x) = x^3 + x + 1$.

1. Write the binary divisor corresponding to $G(x)$.
2. The dataword is 110101. Append the appropriate number of zero bits and perform the modulo-2 division to find the CRC remainder.
3. Write the complete transmitted frame (dataword followed by the remainder).
4. The receiver receives the frame with the third bit flipped. Explain what the receiver computes and what it concludes.

9.3 I prepare for the GCE

EXERCISE 14 **Hamming(7,4) decoding — structured question**

A Hamming(7,4) code is used with parity bits in positions 1, 2 and 4, and data bits in positions 3, 5, 6 and 7. The parity bits are computed with even parity over the following sets of positions:

- p_1 checks positions 1, 3, 5, 7
- p_2 checks positions 2, 3, 6, 7
- p_4 checks positions 4, 5, 6, 7

The received codeword is 1100110 (positions 1 to 7, read left to right).

1. Compute the syndrome bits c_1, c_2 and c_4 for this received word. (3 marks)
2. State, with justification, whether an error occurred during transmission. (2 marks)
3. Locate the position of the faulty bit and give the corrected codeword. (3 marks)
4. Deduce the original 4-bit dataword sent. (2 marks)
5. Explain why this code cannot reliably correct a double-bit error. (2 marks)

EXERCISE 15 **GCE-style essay: choosing a checking method**

A secondary school in Bamenda sends its students' report cards every term from the school server to the regional delegation through a noisy radio link. The IT teacher hesitates between four techniques: single parity bit, two-dimensional parity, additive checksum, CRC, and Hamming code.

1. For each of the four techniques, state: the redundancy (overhead) introduced, its error detection power, and its error correction power. Present your answer in a table. (8 marks)
2. Explain why a simple parity bit is insufficient for this application. (2 marks)
3. The link is noisy but retransmission of a frame is cheap and fast. Recommend a technique for the school and justify your choice with two arguments. (4 marks)
4. Now suppose retransmission is impossible (one-way broadcast). Which technique would you recommend and why? (3 marks)
5. Name one Cameroonian institution or service (e.g. mobile money, ANTIC, CAMTEL) for which reliable data transfer is critical, and explain in two sentences what could happen without error checking. (3 marks)

EXERCISE 16 **Hamming distance and code capability**

Consider the code formed by the four codewords:

$$C_1 = 000000, \quad C_2 = 101010, \quad C_3 = 011110, \quad C_4 = 110100.$$

1. Compute the Hamming distance between each pair of distinct codewords. (3 marks)
2. Deduce the minimum distance d_{min} of this code. (1 mark)
3. State the maximum number of bit errors this code can detect per codeword, justifying with d_{min}. (2 marks)
4. State the maximum number of bit errors this code can correct per codeword, justifying with d_{min}. (2 marks)
5. The word 101011 is received. Which codeword was most probably sent? Justify using the Hamming distance. (3 marks)
6. Give one reason why increasing redundancy improves reliability but is costly in practice. (1 mark)

10 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. Answer: (b) detect any single-bit error. A single parity bit makes the total number of 1s even (or odd). Flipping one bit changes this parity, so the error is detected; but the receiver does not know which bit flipped, so it cannot correct it. Two flipped bits restore the parity, so double-bit errors escape detection.
2. Answer: (c) the number of 1s in the codeword is even. This is the very definition of even parity: the parity bit is chosen (0 or 1) so that the complete codeword, parity bit included, contains an even number of 1s.
3. Answer: (b) polynomial division in modulo-2 arithmetic. The CRC treats the bit string as a polynomial over $\{0, 1\}$ and divides it by a generator polynomial using XOR (modulo-2) arithmetic; the remainder is the check field.
4. Answer: (b) 4 data bits and 3 control bits. In Hamming$(7, 4)$ the codeword has 7 bits in total: 4 data bits and 3 parity (control) bits placed in positions 1, 2, 4.

SOLUTION **Exercise 2 — True or false**

1. False. A single parity bit detects only error patterns that change the parity of the number of 1s, i.e. an odd number of bit flips. Any even number of errors (2, 4, …) leaves the parity unchanged and goes undetected.
2. False. A simple additive checksum sums the data blocks; addition is commutative, so swapping two blocks leaves the sum — and therefore the checksum — unchanged. The swap is not detected.
3. True. A burst error of length L corresponds to an error polynomial of degree less than L. If $L \leq r$ (the degree of the generator), the error polynomial cannot be a multiple of the generator, so the remainder is non-zero and the burst is detected.
4. True. Hamming$(7, 4)$ has minimum distance $d_{min} = 3$, so it corrects $\left\lfloor \frac{3-1}{2} \right\rfloor = 1$ erroneous bit per codeword: the syndrome gives exactly the position of the single flipped bit.

SOLUTION **Exercise 3 — Definitions**

1. Parity bit: an extra bit appended to a binary word, chosen so that the total number of

1s in the transmitted codeword is even (even parity) or odd (odd parity); it allows the detection of an odd number of bit errors.

2. Checksum: a small block of control data obtained by applying an arithmetic operation (typically the sum of the data bytes modulo 2^n) to the message, transmitted with it and recomputed by the receiver to detect alterations.
3. Hamming distance between two codewords: the number of bit positions in which the two words differ, i.e. the number of 1s in their bitwise XOR.
4. Syndrome: the binary number formed by the parity-check results of a received Hamming codeword; a zero syndrome means "no error detected", and a non-zero syndrome gives the position of the erroneous bit.

SOLUTION **Exercise 4 — Direct calculations**

1. The word 1001011 contains four 1s (an even number).
   - Even parity: the count is already even, so the parity bit is $\boxed{0}$; transmitted word: 1001011 0.
   - Odd parity: a 1 must be added to make the count odd, so the parity bit is $\boxed{1}$; transmitted word: 10010111.
2. Compare 1011010 and 1001110 position by position:

Position	1	2	3	4	5	6	7
Word 1	1	0	1	1	0	1	0
Word 2	1	0	0	1	1	1	0
Differ?			×		×		

   They differ in positions 3 and 5 only, so the Hamming distance is $\boxed{d = 2}$.
3. With $d_{min} = 3$: the code can detect up to $d_{min} - 1 = \boxed{2}$ bit errors per codeword, and correct up to $\left\lfloor \frac{d_{min}-1}{2} \right\rfloor = \boxed{1}$ bit error per codeword.

SOLUTION **Exercise 5 — Parity check on received bytes**

1. Count the number of 1s in each received byte (parity bit included); under even parity a valid byte must contain an even number of 1s.

- B_1 = 11001011: five 1s → odd → invalid.
- B_2 = 01010010: three 1s → odd → invalid.
- B_3 = 11100001: four 1s → even → valid.

2. Parity only tells the receiver that the total count of 1s is wrong. Any one of the 8 bits could be the flipped one — flipping any single bit produces exactly the same observed symptom (odd count). The method carries no positional information, so it can only request retransmission.

3. If two bits of a byte are flipped (or any even number), the number of 1s keeps the same parity: the byte is declared valid although it is corrupted. Example: B_3 = 11100001 corrupted into 11000011 (bits 3 and 7 flipped) still has four 1s and passes the even-parity check.

SOLUTION **Exercise 6 — Two-dimensional parity**

1. The received block (4 rows × 7 columns, including row-parity column 7 and column-parity row 4) is:

	Data + Row parity						Row check
1	0	1	1	0	1	0	OK
0	1	0	0	1	0	1	OK
1	1	0	**1**	0	1	1	fails
0	0	1	0	0	1	1	OK
Col check	OK	OK	OK	fails	OK	OK	OK

Recompute the even parity of each row and each column on the block as received (each parity bit must make its line contain an even number of 1s):

Line	Number of 1s (data + parity)	Check
Row 1	4 + 0 = 4	OK
Row 2	3 + 1 = 4	OK
Row 3	4 + 1 = 5	fails
Row 4	3 + 1 = 4	OK

Line	Number of 1s (data + parity)	Check
Col 1	2 + 0 = 2	OK
Col 2	2 + 0 = 2	OK
Col 3	3 + 1 = 4	OK
Col 4	1 + 0 = 1	fails
Col 5	2 + 0 = 2	OK
Col 6	1 + 1 = 2	OK
Col 7	3 + 1 = 4	OK

The only failing row is row 3 and the only failing column is column 4. The erroneous bit is therefore at their intersection: row 3, column 4.

2. The bit received at position (row 3, column 4) is 1; flipping it restores both parities, so the corrected value is $\boxed{0}$.

3. Simple parity mixes all bits of a word into a single check: it signals that something is wrong but not where. Two-dimensional parity gives two independent coordinates of the error: the failing row and the failing column. Their intersection designates a unique bit, which can then be flipped — hence correction of any single-bit error, not mere detection.

SOLUTION **Exercise 7 — Additive checksum**

1. Convert each byte to decimal and sum modulo 256:

$$B_1 = 01100110_2 = 102_{10}, \qquad B_2 = 10101010_2 = 170_{10},$$
$$B_3 = 00011110_2 = 30_{10}, \qquad B_4 = 11000011_2 = 195_{10}.$$
$$S = 102 + 170 + 30 + 195 = 497, \qquad 497 \bmod 256 = 241.$$

The checksum byte is $241_{10} = \boxed{11110001_2}$.

2. The receiver sums the received bytes: $B_2' = 10101000_2 = 168_{10}$, so

$$S' = 102 + 168 + 30 + 195 = 495, \qquad 495 \bmod 256 = 239 \neq 241.$$

The recomputed checksum (239) differs from the received checksum (241): the error is detected and the frame is rejected.

3. Any two corruptions whose effects cancel in the sum are invisible. Example: B_1 arrives as 01100111_2 (103, i.e. +1) and B_2 as 10101001_2 (169, i.e. −1). Then

$$S'' = 103 + 169 + 30 + 195 = 497 \equiv 241 \pmod{256},$$

the checksum matches and the double corruption is not detected.

SOLUTION **Exercise 8 — CRC computation**

1. $G(x) = x^3 + x + 1 = 1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 1$, so the binary divisor is $\boxed{1011}$ (degree $r = 3$).
2. Append $r = 3$ zeros to the dataword: 110101 000. Perform the modulo-2 division by 1011 (XOR, no borrows):

   110101000 ÷ 1011

   1101 ⊕ 1011 = 0110 bring down 0 → 1100
   1100 ⊕ 1011 = 0111 bring down 1 → 1111
   1111 ⊕ 1011 = 0100 bring down 0 → 1000
   1000 ⊕ 1011 = 0011 bring down 0 → 0110
   0110 ⊕ 0000 = 0110 bring down 0 → 1100
   1100 ⊕ 1011 = 0111 no more bits.

   The CRC remainder is $\boxed{111}$.
3. The transmitted frame is dataword + remainder: $\boxed{110101\,111}$.
4. The frame with the third bit flipped is 111101111. The receiver divides the whole received frame by the same divisor 1011:

   1111 ⊕ 1011 = 0100 → bring down 0 → 1000
   1000 ⊕ 1011 = 0011 → bring down 1 → 0111
   0111 ⊕ 0000 = 0111 → bring down 1 → 1111
   1111 ⊕ 1011 = 0100 → bring down 1 → 1001
   1001 ⊕ 1011 = 0010 → bring down 1 → 0101
   0101 ⊕ 0000 = 0101 remainder 101.

   The remainder is 101 ≠ 000: the receiver concludes that the frame is corrupted and rejects it (typically requesting retransmission). Note that the receiver cannot tell which bit flipped — CRC detects but does not correct.

SOLUTION **Exercise 9 — Hamming(7,4) decoding**

We analyse the received word 1100100 (a single-bit corruption of the valid codeword 1100110); the decoding method is identical for any other single-bit error.
Positions: $p_1 = 1$, $p_2 = 1$, $d_3 = 0$, $p_4 = 0$, $d_5 = 1$, $d_6 = 0$, $d_7 = 0$.

1. Syndrome bits (each is 1 if the corresponding set has an odd number of 1s):
   - c_1: positions 1, 3, 5, 7 = 1, 0, 1, 0 → two 1s, even → $c_1 = 0$.

- c_2: positions 2, 3, 6, 7 = 1, 0, 0, 0 → one 1, odd → $c_2 = 1$.
- c_4: positions 4, 5, 6, 7 = 0, 1, 0, 0 → one 1, odd → $c_4 = 1$.

Mark scheme: 1 mark per correct syndrome bit with the positions quoted.

2. The syndrome $c_4c_2c_1 = 110$ is non-zero, so an error did occur during transmission (a zero syndrome would have meant "no error detected"). 2 marks: conclusion 1, justification 1.

3. Reading the syndrome as a binary number, $c_4c_2c_1 = 110_2 = 6$: the faulty bit is at position 6. Flipping it gives the corrected codeword:

$$1100100 \longrightarrow \boxed{1100110}.$$

Check: positions 1, 3, 5, 7 = 1, 0, 1, 0 even; 2, 3, 6, 7 = 1, 0, 1, 0 even; 4, 5, 6, 7 = 0, 1, 1, 0 even. All checks pass. 3 marks: position 1, flip 1, corrected word 1.

4. The data bits occupy positions 3, 5, 6, 7 of the corrected word: 0, 1, 1, 0. The original dataword is $\boxed{0110}$. 2 marks.

5. With two flipped bits the syndrome is again non-zero, but it points to a wrong position (the code has $d_{min} = 3$: a double error moves the sent word to distance 1 from a different valid codeword). "Correcting" that position then introduces a third error. Hamming(7, 4) therefore miscorrects double errors instead of correcting them — it is a single-error-correcting code only. 2 marks.

SOLUTION **Exercise 10 — GCE-style essay: choosing a checking method**

1. Comparison table:

Technique	Redundancy	Detection power	Correction power
Single parity bit	1 bit per word (very low)	Any odd number of bit errors; misses all even numbers of errors	None
Two-dimensional parity	1 bit per row + 1 per column (moderate)	All single-bit errors; many multi-bit patterns	Corrects any single-bit error (row × column intersection)
Additive checksum	1 byte per frame (low)	Most random errors; misses compensating errors and block swaps	None
CRC	r bits per frame (low)	All single, double, odd numbers of errors and all bursts ≤ r bits; extremely reliable	None
Hamming code	3 bits per 4 data bits (high)	All single errors; detects most double errors	Corrects exactly 1 bit per codeword

8 marks: about 2 marks per correct row (overhead, detection, correction).

2. A single parity bit fails whenever an even number of bits is corrupted, and a noisy radio link produces exactly such multi-bit and burst errors. Moreover it gives no way to locate the error. For official documents such as report cards, undetected corruptions are unacceptable. 2 marks.

3. Recommendation: CRC with retransmission (ARQ). Arguments: (i) retransmission is cheap and fast, so detection alone is sufficient — no need to pay for correction capability; (ii) CRC offers by far the best detection power per redundant bit (all bursts up to r bits, all odd numbers of errors) with only a few bits of overhead per frame, unlike Hamming which adds 75% redundancy. 4 marks: choice 1, two justified arguments 3.

4. With one-way broadcast, retransmission is impossible, so the receiver must repair errors itself: a forward error correction code is required — Hamming code (or a stronger FEC code). It lets the receiver correct single-bit errors in each codeword without any feedback channel. 3 marks.

5. Example: mobile money (MTN Mobile Money, Orange Money). Without error checking, a corrupted transaction record could credit the wrong account or the wrong amount, causing direct financial loss to customers and destroying trust in the service. (ANTIC securing e-government exchanges or CAMTEL billing data are equally acceptable answers.) 3 marks: named service 1, consequence explained 2.

SOLUTION **Exercise 11 — Hamming distance and code capability**

1. The four codewords (6 bits each) are:

$$C_1 = 000000, \quad C_2 = 101010, \quad C_3 = 011110, \quad C_4 = 110100.$$

Distances between all pairs (count of differing positions, i.e. number of 1s in the XOR):

Pair	XOR	Distance
C_1, C_2	101010	3
C_1, C_3	011110	4
C_1, C_4	110100	3
C_2, C_3	110100	3
C_2, C_4	011110	4
C_3, C_4	101010	3

3 marks: $\frac{1}{2}$ mark per correct distance.

2. The minimum distance is the smallest of these values: $\boxed{d_{min} = 3}$. 1 mark.

3. A code with minimum distance d_{min} detects up to $d_{min} - 1$ errors, because fewer than d_{min} flips can never transform one valid codeword into another valid one. Here: up to $\boxed{2}$ bit errors detected per codeword. 2 marks: value 1, justification 1.

4. Correction requires the received word to remain closer to the sent codeword than to any other: up to $\left\lfloor \frac{d_{min}-1}{2} \right\rfloor = \boxed{1}$ bit error corrected per codeword. 2 marks.

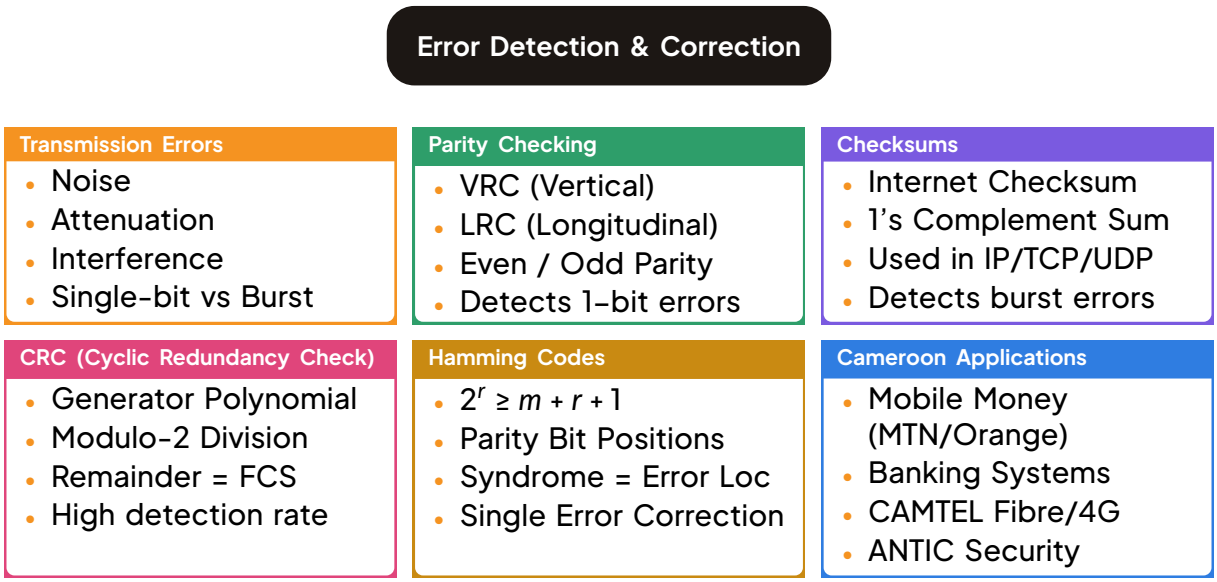
5. Compute the distance from $R = 101011$ to each codeword:

	C_1	C_2	C_3	C_4
$d(R, C_i)$	4	1	4	5

The nearest codeword is $C_2 = 101010$, at distance only 1 (they differ only in the last bit). By the nearest-neighbour (maximum-likelihood) principle, the most probably sent codeword is $\boxed{C_2 = 101010}$. 3 marks: distances 1, choice 1, justification 1.

6. Each extra redundant bit consumes bandwidth (or storage) and transmission time without carrying information: increasing redundancy lowers the effective data rate and raises cost, so a compromise between reliability and efficiency must be found. 1 mark.

11 SUMMARY SHEET



Chapter ICT11 Mind Map: Error Detection and Correction Methods

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key Definitions

- **Transmission Error**: Alteration of bits during transfer due to noise, attenuation, or interference. **Single-bit error**: one bit flipped. **Burst error**: two or more consecutive bits flipped.
- **Redundancy**: Extra bits added to data to enable detection/correction. **Codeword** = Data + Redundancy.
- **Hamming Distance**: Number of differing bits between two codewords. Minimum distance d_{min} determines capability: detects $d_{min} - 1$ errors, corrects $\lfloor (d_{min} - 1)/2 \rfloor$ errors.
- **FCS (Frame Check Sequence)**: Redundancy field appended to a frame (e.g., CRC remainder).

Parity Checking (Lesson 35 Core)

- **VRC (Vertical Redundancy Check)**: One parity bit per character/byte. **Even parity**: total 1s even. **Odd parity**: total 1s odd. Detects all single-bit errors; fails if even number of bits flipped.

- **LRC (Longitudinal Redundancy Check)**: Parity bit computed for each bit position across a block of characters (column-wise). Often sent as a Block Check Character (BCC). Combined with VRC (2D parity) detects burst errors better.

Checksums

- Used in Internet Layer (IP, TCP, UDP). Sender divides data into k-bit segments (usually 16-bit).
- **Algorithm**: Sum all segments using **1's complement arithmetic** (wrap-around carry added to LSB). Take 1's complement of sum → **Checksum**.
- Receiver repeats sum including checksum; result should be all 1s (zero in 1's complement). Detects many burst errors but not all (e.g., compensating errors).

Cyclic Redundancy Check (CRC) – Most Powerful Detection

- Based on **Polynomial Division** in $GF(2)$ (Modulo-2 arithmetic: XOR for add/sub).
- **Generator Polynomial** $G(x)$: Agreed standard (e.g., CRC-16: $x^{16} + x^{15} + x^2 + 1$, CRC-32 Ethernet).
- **Steps**:
  1. Append r zeros to data (r = degree of $G(x)$).
  2. Perform Modulo-2 division of augmented data by $G(x)$.
  3. **Remainder** $R(x)$ **(FCS)** replaces the r zeros → Transmitted Codeword $T(x)$.
- Receiver divides $T(x)$ by $G(x)$; **Remainder = 0** ⇒ Accept (high probability), else Reject.
- Detects: All single-bit, all double-bit (if $G(x)$ has 3+ terms), all odd errors (if $G(x)$ has factor $x + 1$), all bursts ≤ r, most longer bursts.

Hamming Codes – Single Error Correction (SEC)

- **Redundancy bits** r for m data bits: $2^r \geq m + r + 1$.
- **Positions**: Parity bits at powers of 2 (1, 2, 4, 8…). Data bits fill gaps.
- **Parity Calculation**: Each parity bit P_i covers positions where binary index has bit i set (even parity standard).
- **Syndrome Calculation** (Receiver): Recompute parity checks C_i. Syndrome $S = C_r \ldots C_2 C_1$ (binary).
  - $S = 0$ ⇒ No error.
  - $S \neq 0$ ⇒ Binary value = position of corrupted bit. Flip it to correct.
- Extended Hamming (SECDED): Adds overall parity bit for Double Error Detection.

Common Mistakes & Traps

- Confusing VRC and LRC: VRC is row-wise (per byte), LRC is column-wise (per bit position across block).
- CRC Arithmetic: Using decimal/subtraction instead of **XOR (Modulo-2)**. No borrow/carry!
- Generator Polynomial Degree: Appending wrong number of zeros (must be degree of $G(x)$, not number of bits in $G(x)$).
- Hamming Syndrome Order: C_1 is LSB of syndrome (position 1), C_2 is next... Reversing order gives wrong location.
- Detection vs Correction: Parity/Checksum/CRC detect only (request retransmission/ARQ). Hamming corrects (Forward Error Correction/FEC). Do not claim CRC corrects errors.

GCE Examination Tips

- Show Working: For CRC, draw the Modulo-2 division tableau clearly. For Hamming, show bit positions table and syndrome calculation steps.
- Define Terms: "Define Hamming Distance", "What is a Generator Polynomial?" are frequent 2-3 mark questions.
- Compare Methods: Table comparing VRC, LRC, Checksum, CRC, Hamming on: Error Detection Capability, Correction Capability, Overhead, Complexity, Typical Use Case.
- Contextualise: Mention **Mobile Money (MTN MoMo, Orange Money)** uses encryption + CRC/Checksums for transaction integrity. **ANTIC** promotes secure protocols (TLS/SSL) using these checks. **CAMTEL** fibre links use CRC at Data Link Layer (Ethernet frames).
- Formula Recall: $2^r \geq m + r + 1$ for Hamming bits. Know standard CRC polynomials (CRC-16, CRC-CCITT, CRC-32).

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS
OnBuch+